CRYPTO 2
Cours 1 - Signatures numériques
Signatures de Lamport

Damien Vergnaud

Sorbonne Université – CNRS

Contents

1. Introduction
   - Definitions
   - Security Notions for Digital Signatures
2. Lamport signatures
   - Signing one bit
   - Signing k bits
3. Lamport signatures - Variants
   - Signing $\leq k$ bits
   - Short secret/verification keys
4. Groth's one-time signatures
5. SPHINCS
   - Optimized Lamport signatures
   - Winternitz signatures
   - HORS signatures
   - SPHINCS Signatures

Why "Provable Security" ?

Once a cryptosystem is described, how can we prove its security?

by trying to exhibit an attack

- attack found
  $\Rrightarrow$ system insecure!
- attack not found
  $\Rrightarrow$?

by proving that no attack exists under some assumptions

- attack found
  $\Rrightarrow$ false assumption

- **"Textbook" cryptosystems cannot be used as such**
- **Pratictioners need formatting rules to ensure operability.**
  $\rightsquigarrow$ Paddings are used in practice : heuristic security
- **Provable security is needed in upcoming systems.**
  This is no longer just theory.

Why "Provable Security" ?

Once a cryptosystem is described, how can we prove its security?

by trying to exhibit an attack

- attack found
  $\Rrightarrow$ system insecure!
- attack not found
  $\Rrightarrow$?

by proving that no attack exists under some assumptions

- attack found
  $\Rrightarrow$ false assumption

- **"Textbook" cryptosystems cannot be used as such**
- **Pratictioners need formatting rules to ensure operability.**
  $\rightsquigarrow$ Paddings are used in practice : heuristic security
- **Provable security is needed in upcoming systems.**
  This is no longer just theory.

Why "Provable Security" ?

Once a cryptosystem is described, how can we prove its security?

by trying to exhibit an attack

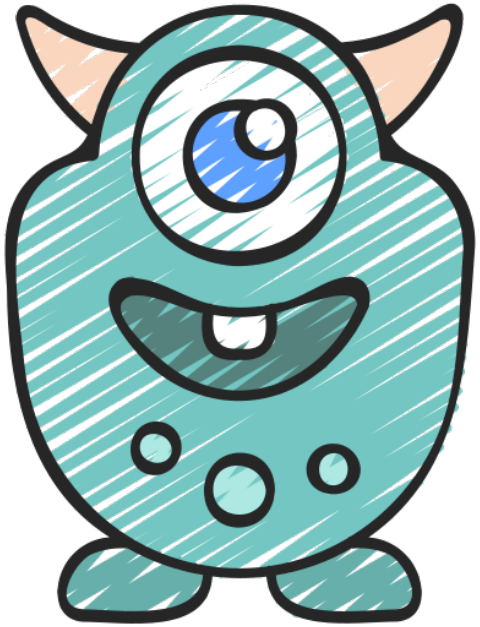
- attack found
  $\Rrightarrow$ system insecure!
- attack not found
  $\Rrightarrow$?

by proving that no attack exists under some assumptions

- attack found
  $\Rrightarrow$ false assumption

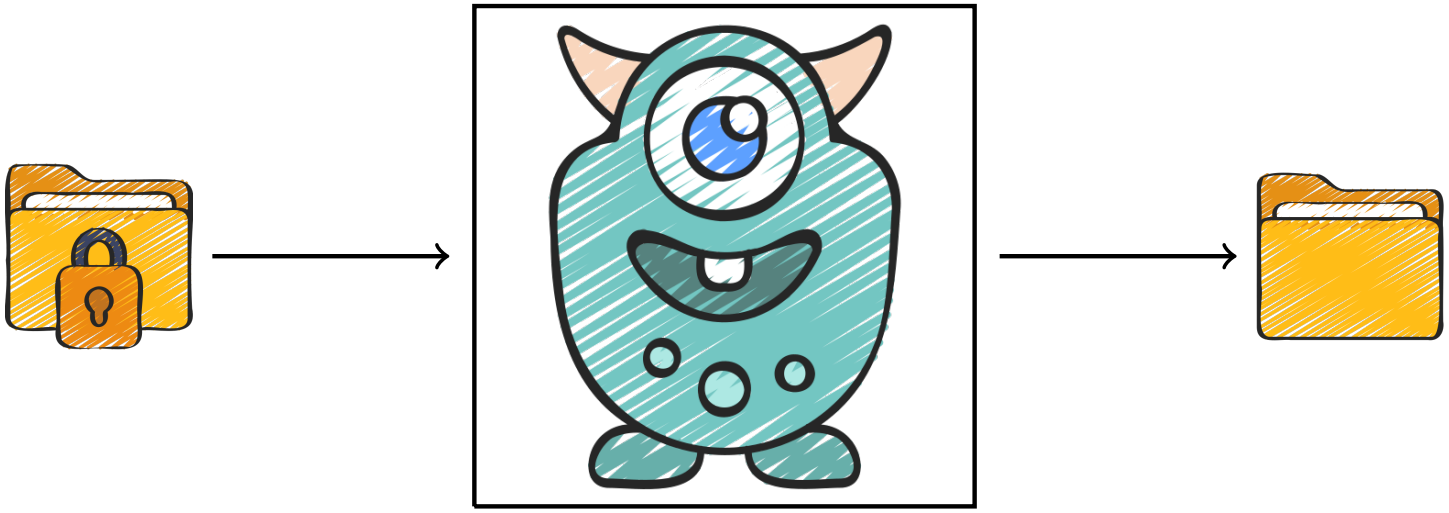
- **"Textbook" cryptosystems cannot be used as such**
- **Pratictioners need formatting rules to ensure operability.**
  $\rightsquigarrow$ Paddings are used in practice : heuristic security
- **Provable security is needed in upcoming systems.**
  This is no longer just theory.

Who is the adversary?



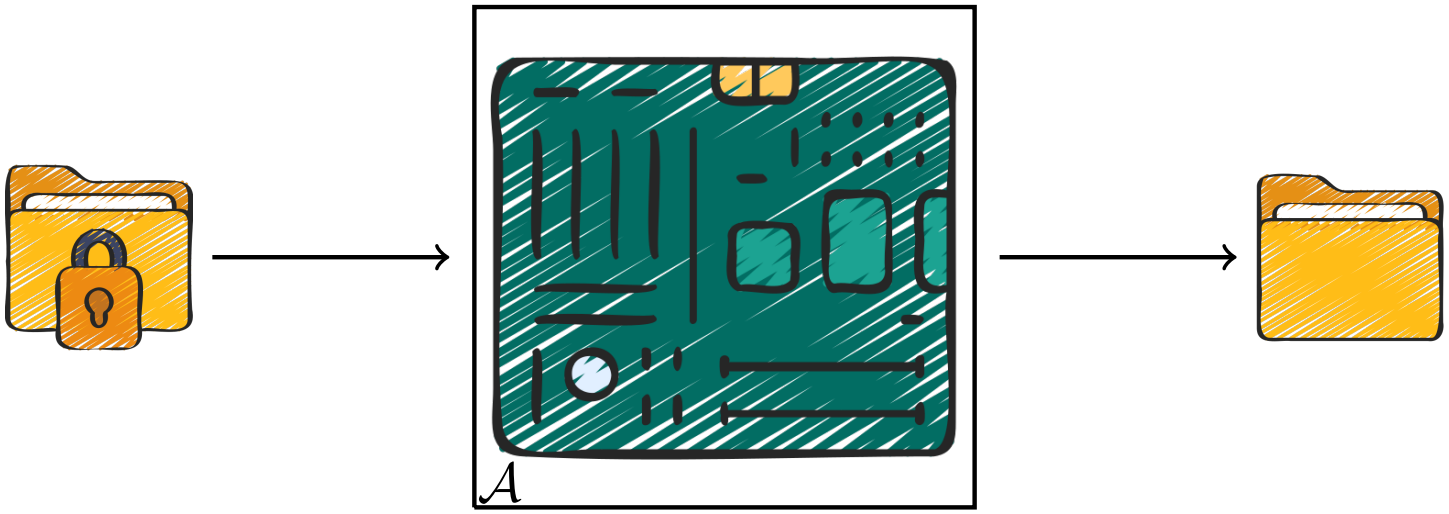
- a probabilistic polynomial time Turing machine (PPTM) **(Church-Turing thesis)**
- knows all the algorithms **(Kerckoff's principles)**
- has full access to communication media

Proof by reduction



$\mathcal{A}$ adversary against e.g. one-wayness

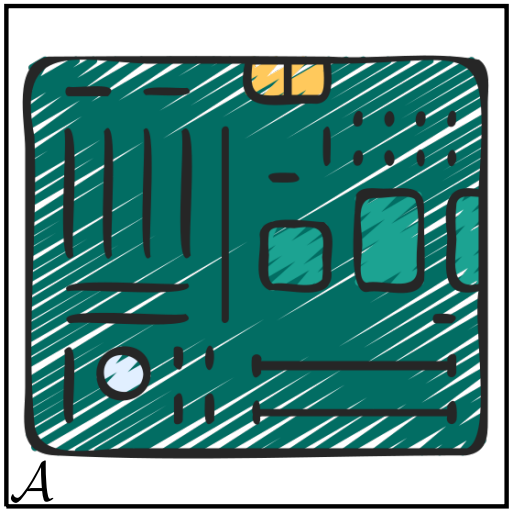
Proof by reduction



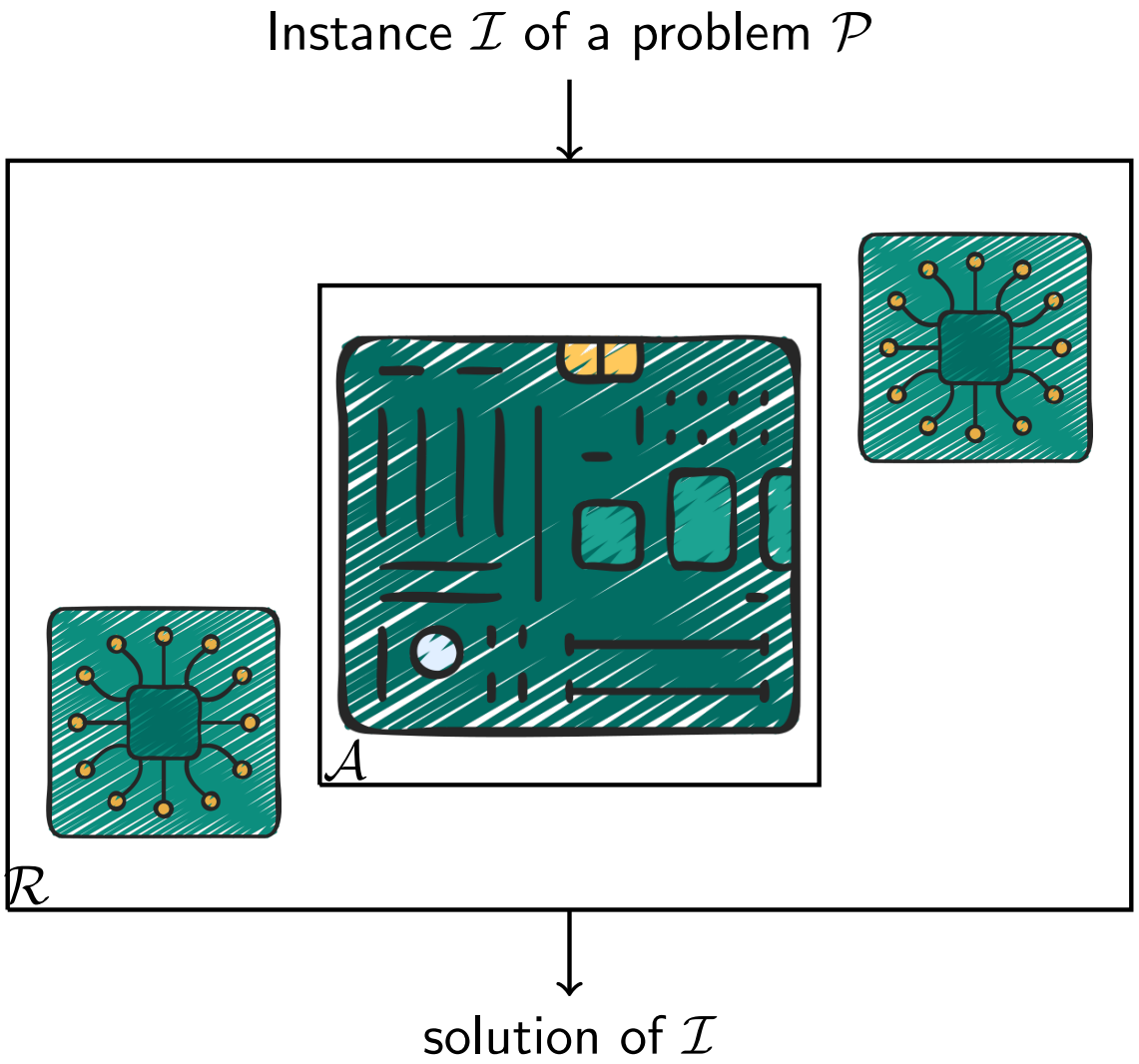
$\mathcal{A}$ adversary against e.g. one-wayness

Proof by reduction

Instance $\mathcal{I}$ of a problem $\mathcal{P}$



Proof by reduction



The Methodology of "Provable Security"

1. Define goal of adversary
2. Define security model
3. Define complexity assumptions
4. Provide a proof by reduction
5. Check proof
6. Interpret proof

The Methodology of "Provable Security"

1. Define goal of adversary
2. Define security model
3. Define complexity assumptions
4. Provide a proof by reduction
5. Check proof
6. Interpret proof

Digital Signatures

- A very important public key primitive is the **digital signature**.
- The idea is
  - Message + Alice's Private Key = Signature
  - Message + Signature + Alice's Public Key = YES/NO
- Alice can sign a message using her private/signing key.
- Anyone can verify Alice's signature, since everyone can obtain her public/verification key.
- the verifier is convinced that only Alice could have produced the signature
  - **only Alice knows her private key!**

Digital Signatures

- A very important public key primitive is the **digital signature**.
- The idea is
  - Message + Alice's Private Key = Signature
  - Message + Signature + Alice's Public Key = YES/NO
- Alice can sign a message using her private/signing key.
- Anyone can verify Alice's signature, since everyone can obtain her public/verification key.
- the verifier is convinced that only Alice could have produced the signature
  - **only Alice knows her private key!**

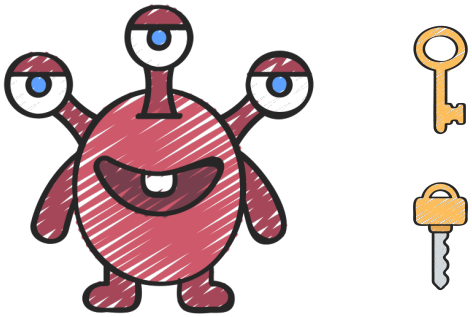
Digital Signatures

- A very important public key primitive is the **digital signature**.
- The idea is
  - Message + Alice's Private Key = Signature
  - Message + Signature + Alice's Public Key = YES/NO
- Alice can sign a message using her private/signing key.
- Anyone can verify Alice's signature, since everyone can obtain her public/verification key.
- the verifier is convinced that only Alice could have produced the signature
  - **only Alice knows her private key!**

Digital Signatures

- A very important public key primitive is the **digital signature**.
- The idea is
  - Message + Alice's Private Key = Signature
  - Message + Signature + Alice's Public Key = YES/NO
- Alice can sign a message using her private/signing key.
- Anyone can verify Alice's signature, since everyone can obtain her public/verification key.
- the verifier is convinced that only Alice could have produced the signature
  - **only Alice knows her private key!**

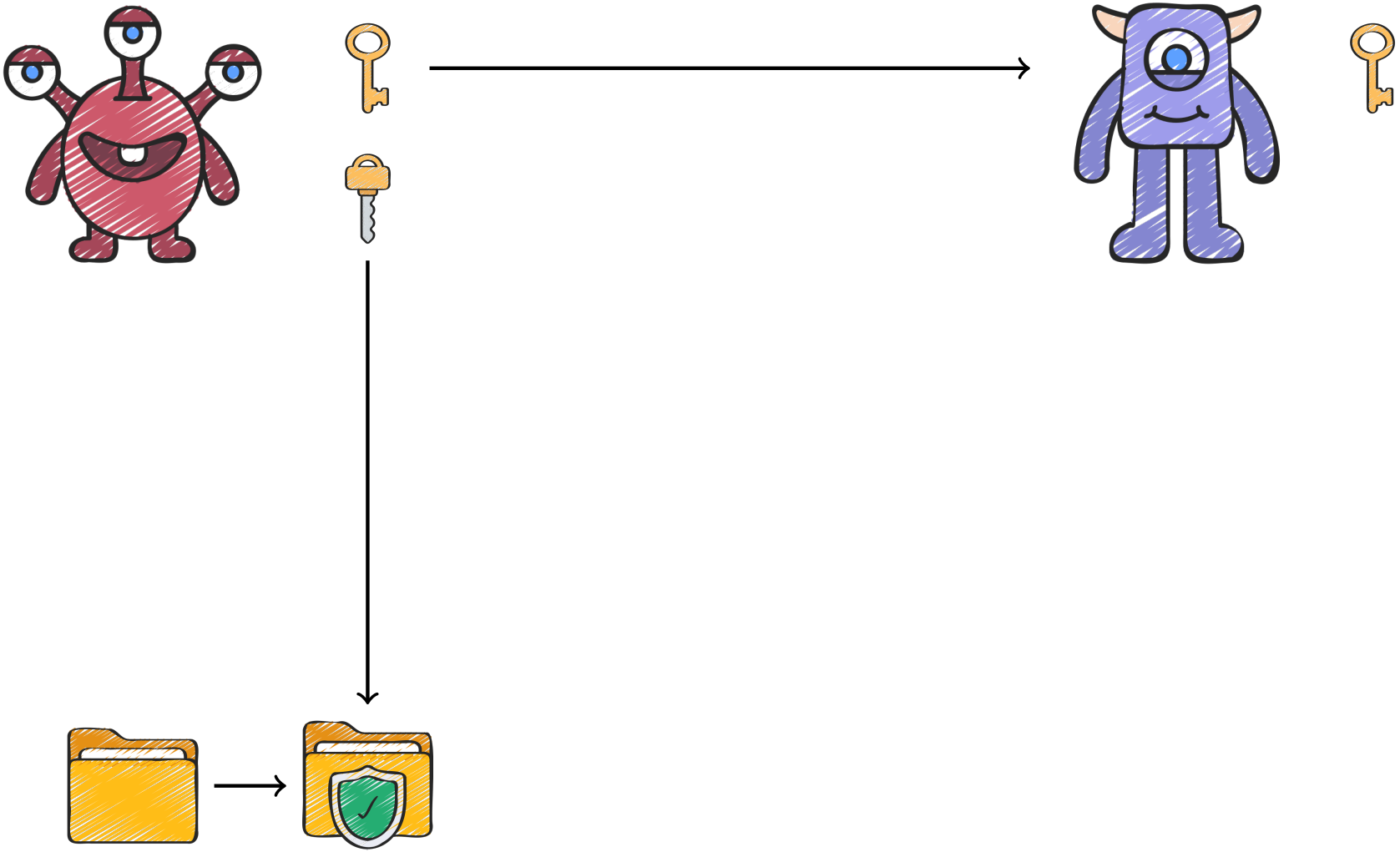
Digital signature schemes



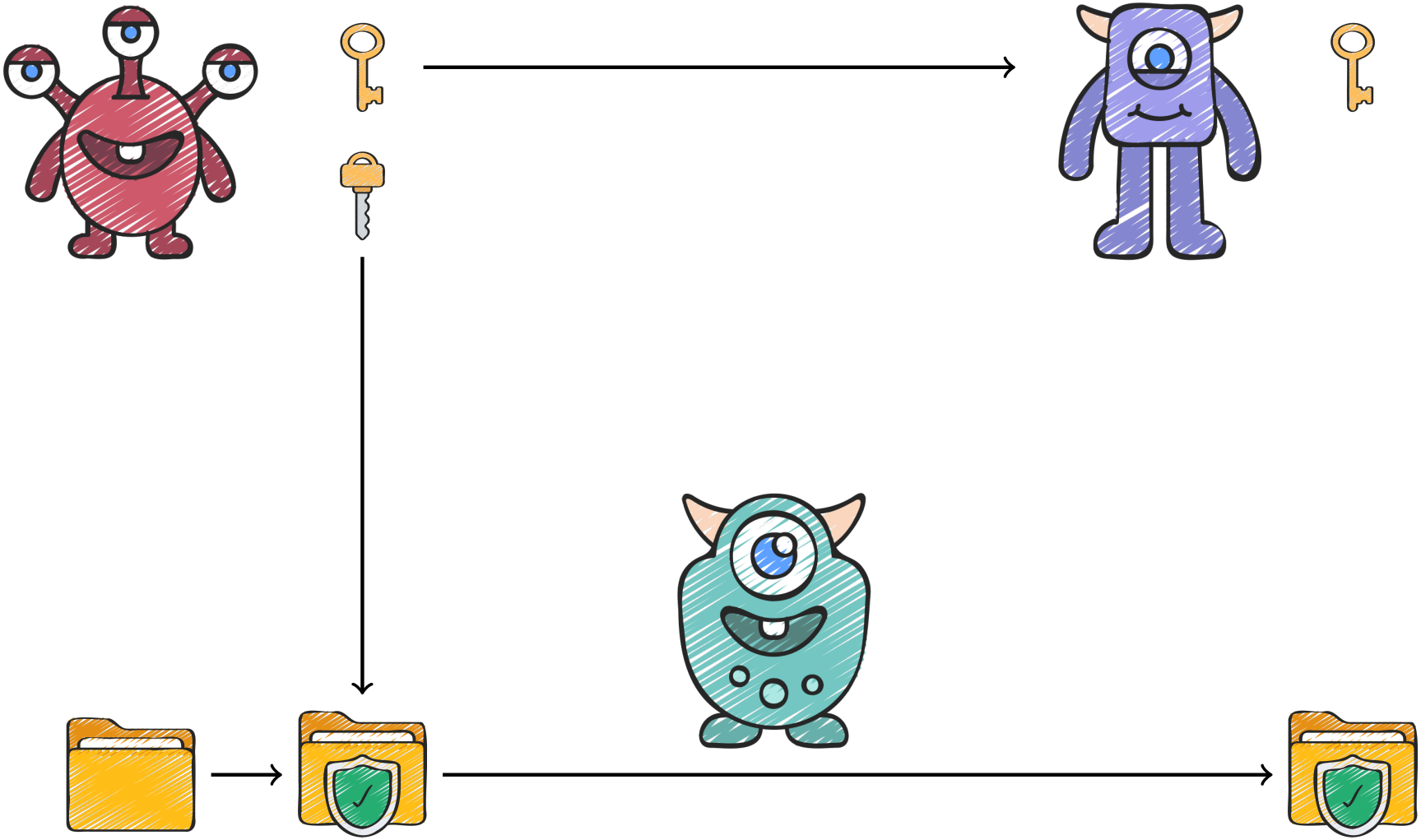
Digital signature schemes

Digital signature schemes

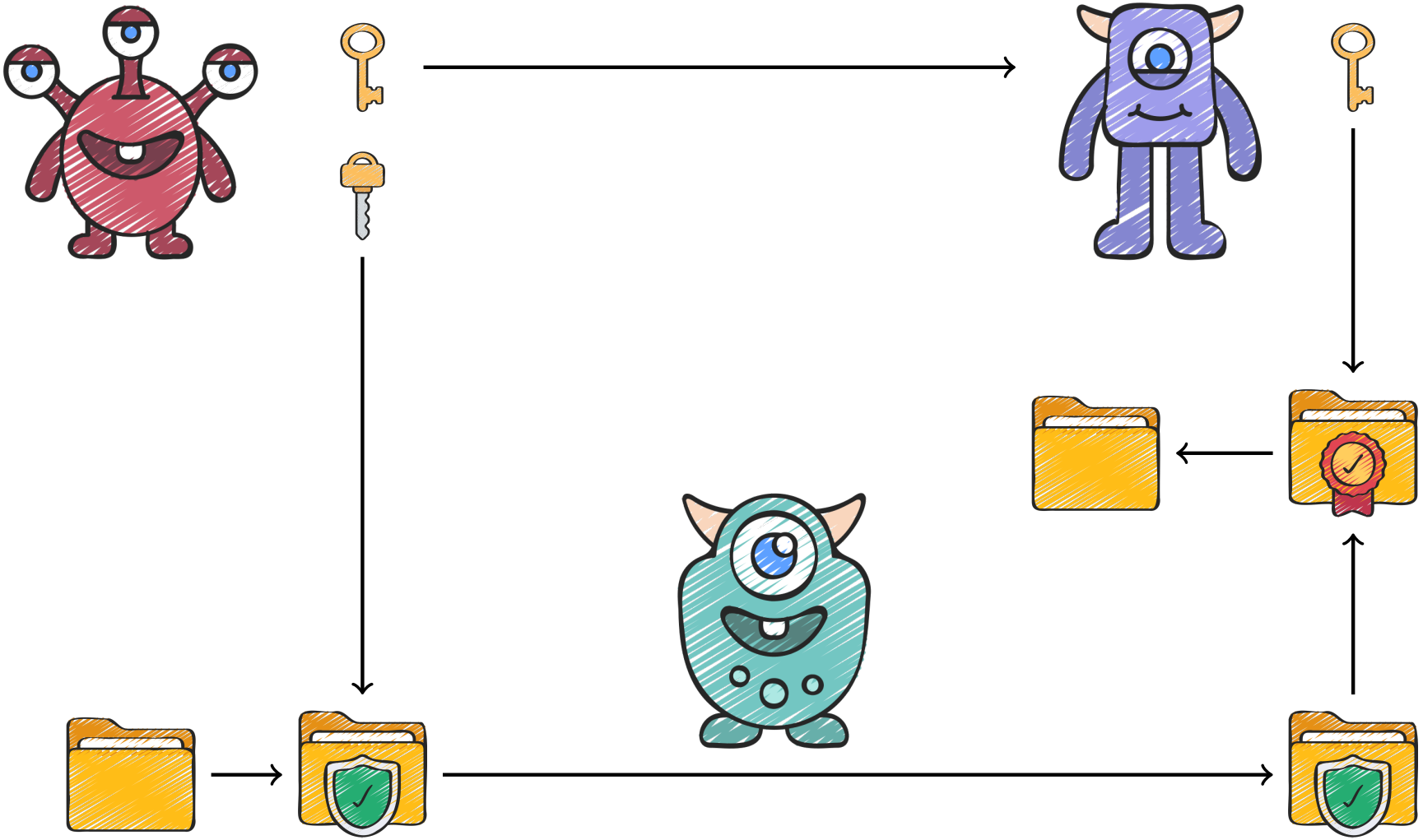
Digital signature schemes



Digital signature schemes



Digital signature schemes



Digital Signatures : Services

- The verification algorithm is used to determine whether or not the signature is properly constructed.
- the verifier has guarantee of
  - message integrity and
  - message origin.
- also provide non-repudiation - not provided by MACs.

Most important cryptographic primitive!

Security Notions

Depending on the context in which a given cryptosystem is used, one may formally define a security notion for this system,

- by telling what **goal** an adversary would attempt to reach,
- and what means or information are made available to her (the **attack model**).

A security notion (or level) is entirely defined by pairing an adversarial goal with an adversarial model.

Examples: UB-KMA, UUF-KOA, EUF-CMA.

Signature Schemes

An digital signature scheme is a triple of algorithms $(\mathcal{K}, \mathcal{S}, \mathcal{V})$ where

- $\mathcal{K}$ is a probabilistic **key generation algorithm** which returns random pairs of secret and verification keys (sk, vk) depending on the security parameter κ,
- $\mathcal{S}$ is a (probabilistic) **signature algorithm** which takes on input a signing key sk and a *message* $m \in \mathcal{M}$, runs on a random tape $u \in \mathcal{U}$ and returns $s \in S$,
- $\mathcal{V}$ is a deterministic **verification algorithm** which takes on input a verification key vk, a message m and $s \in S$ and outputs a bit in $\{0, 1\}$. If $\mathcal{V}_{vk}(m, s) = 1$, then s is a *signature* on m for vk.

If $(sk, vk) \leftarrow \mathcal{K}$, then $\mathcal{V}_{vk}(m, \mathcal{S}_{sk}(m, u)) = 1$ for all $(m, u) \in \mathcal{M} \times \mathcal{U}$.

Signature Schemes

An digital signature scheme is a triple of algorithms $(\mathcal{K}, \mathcal{S}, \mathcal{V})$ where

- $\mathcal{K}$ is a probabilistic **key generation algorithm** which returns random pairs of secret and verification keys (sk, vk) depending on the security parameter κ,
- $\mathcal{S}$ is a (probabilistic) **signature algorithm** which takes on input a signing key sk and a *message* $m \in \mathcal{M}$, runs on a random tape $u \in \mathcal{U}$ and returns $s \in S$,
- $\mathcal{V}$ is a deterministic **verification algorithm** which takes on input a verification key vk, a message m and $s \in S$ and outputs a bit in $\{0, 1\}$. If $\mathcal{V}_{vk}(m, s) = 1$, then s is a *signature* on m for vk.

If $(sk, vk) \leftarrow \mathcal{K}$, then $\mathcal{V}_{vk}(m, \mathcal{S}_{sk}(m, u)) = 1$ for all $(m, u) \in \mathcal{M} \times \mathcal{U}$.

Signature Schemes

An digital signature scheme is a triple of algorithms $(\mathcal{K}, \mathcal{S}, \mathcal{V})$ where

- $\mathcal{K}$ is a probabilistic **key generation algorithm** which returns random pairs of secret and verification keys (sk, vk) depending on the security parameter κ,
- $\mathcal{S}$ is a (probabilistic) **signature algorithm** which takes on input a signing key sk and a *message* $m \in \mathcal{M}$, runs on a random tape $u \in \mathcal{U}$ and returns $s \in S$,
- $\mathcal{V}$ is a deterministic **verification algorithm** which takes on input a verification key vk, a message m and $s \in S$ and outputs a bit in $\{0, 1\}$. If $\mathcal{V}_{vk}(m, s) = 1$, then s is a *signature* on m for vk.

If $(sk, vk) \leftarrow \mathcal{K}$, then $\mathcal{V}_{vk}(m, \mathcal{S}_{sk}(m, u)) = 1$ for all $(m, u) \in \mathcal{M} \times \mathcal{U}$.

Security Goals

[Unbreakability] the attacker recovers the secret key *sk* from the verification key *vk* (or an equivalent key if any). This goal is denoted **UB**. Implicitly appeared with public-key cryptography.

[Universal Unforgeability] the attacker, without necessarily having recovered *sk*, can produce a valid signature of any message in the message space. Noted **UUF**.

[Existential Unforgeability] the attacker creates a message and a valid signature of it (likely not of his choosing). Denoted **EUF**.

Adversarial Models

- **Key-Only Attacks** (KOA), unavoidable scenario.
- **Known Message Attacks** (KMA) where an adversary has access to signatures for a set of known messages.
- **Chosen-Message Attacks** (CMA) the adversary is allowed to use the signer as an oracle (full access), and may request the signature of any message of his choice

Chosen-Message Security

Goldwasser, Micali, Rivest (1988)
A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks.
SIAM J. Comput. 17(2) pp. 281-308.

Formally, an signature scheme is said to be (q, τ, ε)-secure if for any adversary $\mathcal{A}$ with running time upper-bounded by τ,

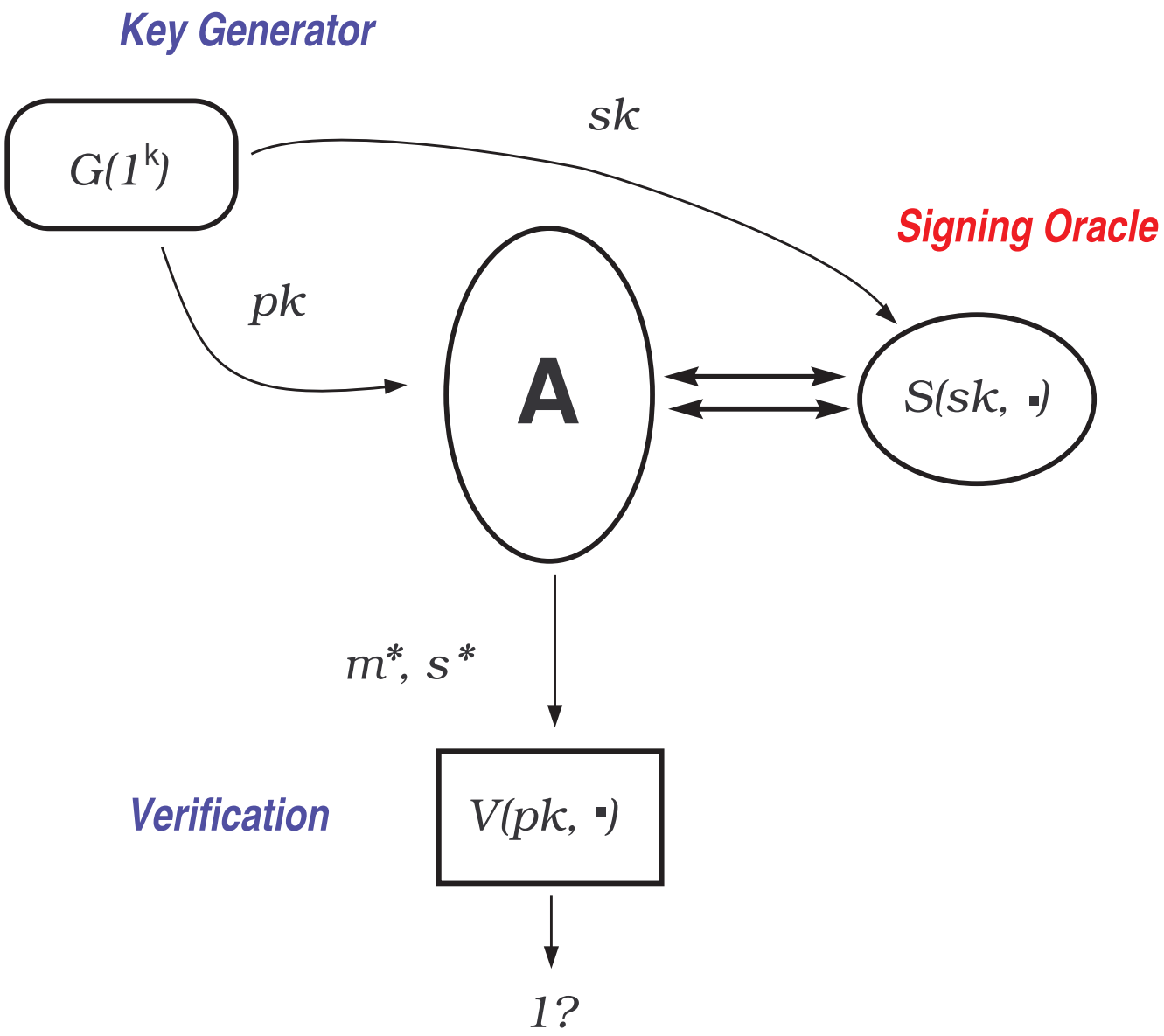
$$\mathrm{Succ}^{\mathrm{EUF-CMA}}(\mathcal{A}) = \Pr\left[\begin{array}{l} (sk, vk) \leftarrow \mathcal{K}(1^{\mathsf{k}}), \\ (m^*, s^*) \leftarrow \mathcal{A}^{\mathcal{S}(sk,\cdot)}(vk), \\ \mathcal{V}(vk, m^*, s^*) = 1 \end{array}\right] < \varepsilon \ ,$$

where the probability is taken over all random choices.

The notation $\mathcal{A}^{\mathcal{S}(sk,\cdot)}$ means that the adversary has access to a signing oracle throughout the game, but at most q times.

The message m^* output by $\mathcal{A}$ was **never** requested to the signing oracle...

EUF-CMA: Playing the Game



Outline

1. Introduction
   - Definitions
   - Security Notions for Digital Signatures
2. Lamport signatures
   - Signing one bit
   - Signing k bits
3. Lamport signatures - Variants
   - Signing $\leq k$ bits
   - Short secret/verification keys
4. Groth's one-time signatures
5. SPHINCS
   - Optimized Lamport signatures
   - Winternitz signatures
   - HORS signatures
   - SPHINCS Signatures

Lamport signatures

L. Lamport
Constructing digital signatures from a one-way function
Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, Oct. 1979.

- a Lamport signature or **Lamport one-time signature scheme** is a method for constructing efficient digital signatures.
- Lamport signatures can be built from any cryptographically secure **one-way** function; usually a **cryptographic hash function** is used.
- Unfortunately each Lamport key can only be used to sign a single message.
- However, we will see how a single key could be used for **many** messages.

One-way function

- A **one-way function** f is a function that is "easy to compute" but "hard to invert"
- Formally, a function $f : X \to Y$ is said to be (τ, ε)-one-way if
  - (Easy to compute) there exist a PT Eval s.t. $y \leftarrow \mathsf{Eval}(x) \Rightarrow y = f(x)$
  - (Hard to invert) for any PPT $\mathcal{A}$ with running time upper-bounded by τ,

$$\mathsf{Succ}^{\mathsf{OW}}(\mathcal{A}) = \Pr\left[\begin{array}{l} x \leftarrow X, \\ x' \leftarrow \mathcal{A}(\mathsf{Eval}(x)), \\ \mathsf{Eval}(x') = \mathsf{Eval}(x) \end{array}\right] < \varepsilon \ ,$$

  where the probability is taken over all random choices (*i.e.* x and $\mathcal{A}$'s random tape).

- Note that unlike hardness in most of complexity theory (e.g., NP-hardness), "hard" in the context of one-way functions refers to **average-case** hardness rather than **worst-case** hardness.

How to sign **one** bit **just once** ?

$\mathcal{M} = \{0, 1\}$

- **Key generation:**
  - Consider $f : X \longrightarrow Y$ a **one-way function**.
    e.g.
    $$\begin{array}{rccc} f: & \mathbb{Z}_q & \longrightarrow & \mathbb{G} \\ & x & \longmapsto & f(x) = g^x \end{array}$$
  - Select two random elements $x_0, x_1 \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in \{0, 1\}$.

  Verification key $vk = (y_0, y_1)$ which can be published.
  Signing key $sk = (x_0, x_1)$ which needs to be kept secret

- **Signature:** if Alice wants to sign a bit b, she does the following:
  - Use her signing key (x_0, x_1) to send the signature $s = x_b$ to Bob.
- **Verification:** to check the validity of s on b, Bob does the following:
  - Obtain Alice's authentic verification key (y_0, y_1).
  - Check whether $f(s) = y_b$.

How to sign **one** bit **just once** ?

$\mathcal{M} = \{0, 1\}$

- **Key generation:**
  - Consider $f : X \longrightarrow Y$ a **one-way function**.
    e.g.
    $$\begin{array}{rccc} f: & \mathbb{Z}_q & \longrightarrow & \mathbb{G} \\ & x & \longmapsto & f(x) = g^x \end{array}$$
  - Select two random elements $x_0, x_1 \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in \{0, 1\}$.

  Verification key $vk = (y_0, y_1)$ which can be published.
  Signing key $sk = (x_0, x_1)$ which needs to be kept secret

- **Signature:** if Alice wants to sign a bit b, she does the following:
  - Use her signing key (x_0, x_1) to send the signature $s = x_b$ to Bob.
- **Verification:** to check the validity of s on b, Bob does the following:
  - Obtain Alice's authentic verification key (y_0, y_1).
  - Check whether $f(s) = y_b$.

How to sign **one** bit **just once** ?

$\mathcal{M} = \{0, 1\}$

- **Key generation:**
  - Consider $f : X \longrightarrow Y$ a **one-way function**.
    e.g.
    $$\begin{array}{rccc} f: & \mathbb{Z}_q & \longrightarrow & \mathbb{G} \\ & x & \longmapsto & f(x) = g^x \end{array}$$
  - Select two random elements $x_0, x_1 \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in \{0, 1\}$.

  Verification key $vk = (y_0, y_1)$ which can be published.
  Signing key $sk = (x_0, x_1)$ which needs to be kept secret

- **Signature:** if Alice wants to sign a bit b, she does the following:
  - Use her signing key (x_0, x_1) to send the signature $s = x_b$ to Bob.
- **Verification:** to check the validity of s on b, Bob does the following:
  - Obtain Alice's authentic verification key (y_0, y_1).
  - Check whether $f(s) = y_b$.

How to sign k bits **just once** ?

$$\boxed{\mathcal{M} = \{0,1\}^k}$$

- **Key generation:**
  - Generate $f : X \longrightarrow Y$ a **one-way function**.
  - Select $2k$ random elements $x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k} \in X$.
  - Compute their images $y_{i,j} = f(x_{i,j})$ for $i \in \{0,1\}$ and $j \in [\![1,k]\!]$.

  Verification key $vk = (y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$ which can be published.
  Signing key $sk = (x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ which needs to be kept secret

- **Signature:** if Alice wants to sign $m = m_1 \ldots m_k$, she does the following:
  - Use her signing key $(x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ to send the signature $s = (x_{m_1,1}, x_{m_1,2}, \ldots, x_{m_k,k})$ to Bob.
- **Verification:** to check the validity of $s = (s_1, \ldots, s_k)$ on m, Bob does the following:
  - Obtain Alice's authentic verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$.
  - Check whether $f(s_i) = y_{m_b,i}$ for all $i \in [\![1,k]\!]$.

How to sign k bits **just once** ?

$$\boxed{\mathcal{M} = \{0,1\}^k}$$

- **Key generation:**
  - Generate $f : X \longrightarrow Y$ a **one-way function**.
  - Select $2k$ random elements $x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k} \in X$.
  - Compute their images $y_{i,j} = f(x_{i,j})$ for $i \in \{0,1\}$ and $j \in [\![1,k]\!]$.

  Verification key $vk = (y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$ which can be published.
  Signing key $sk = (x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ which needs to be kept secret

- **Signature:** if Alice wants to sign $m = m_1 \ldots m_k$, she does the following:
  - Use her signing key $(x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ to send the signature $s = (x_{m_1,1}, x_{m_1,2}, \ldots, x_{m_k,k})$ to Bob.
- **Verification:** to check the validity of $s = (s_1, \ldots, s_k)$ on m, Bob does the following:
  - Obtain Alice's authentic verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$.
  - Check whether $f(s_i) = y_{m_b,i}$ for all $i \in [\![1,k]\!]$.

How to sign k bits **just once** ?

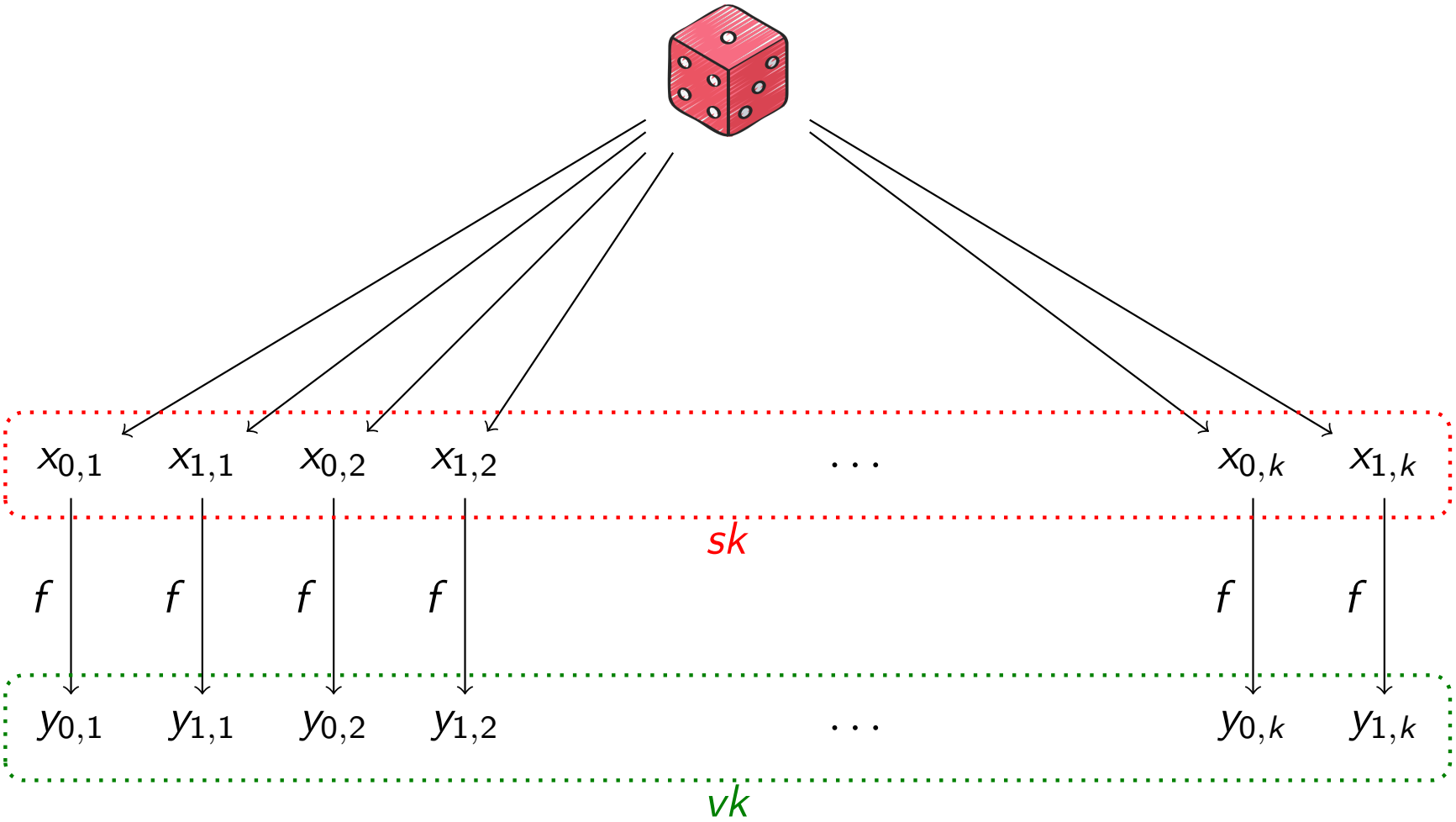
$$\boxed{\mathcal{M} = \{0,1\}^k}$$

- **Key generation:**
  - Generate $f : X \longrightarrow Y$ a **one-way function**.
  - Select $2k$ random elements $x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k} \in X$.
  - Compute their images $y_{i,j} = f(x_{i,j})$ for $i \in \{0,1\}$ and $j \in [\![1,k]\!]$.

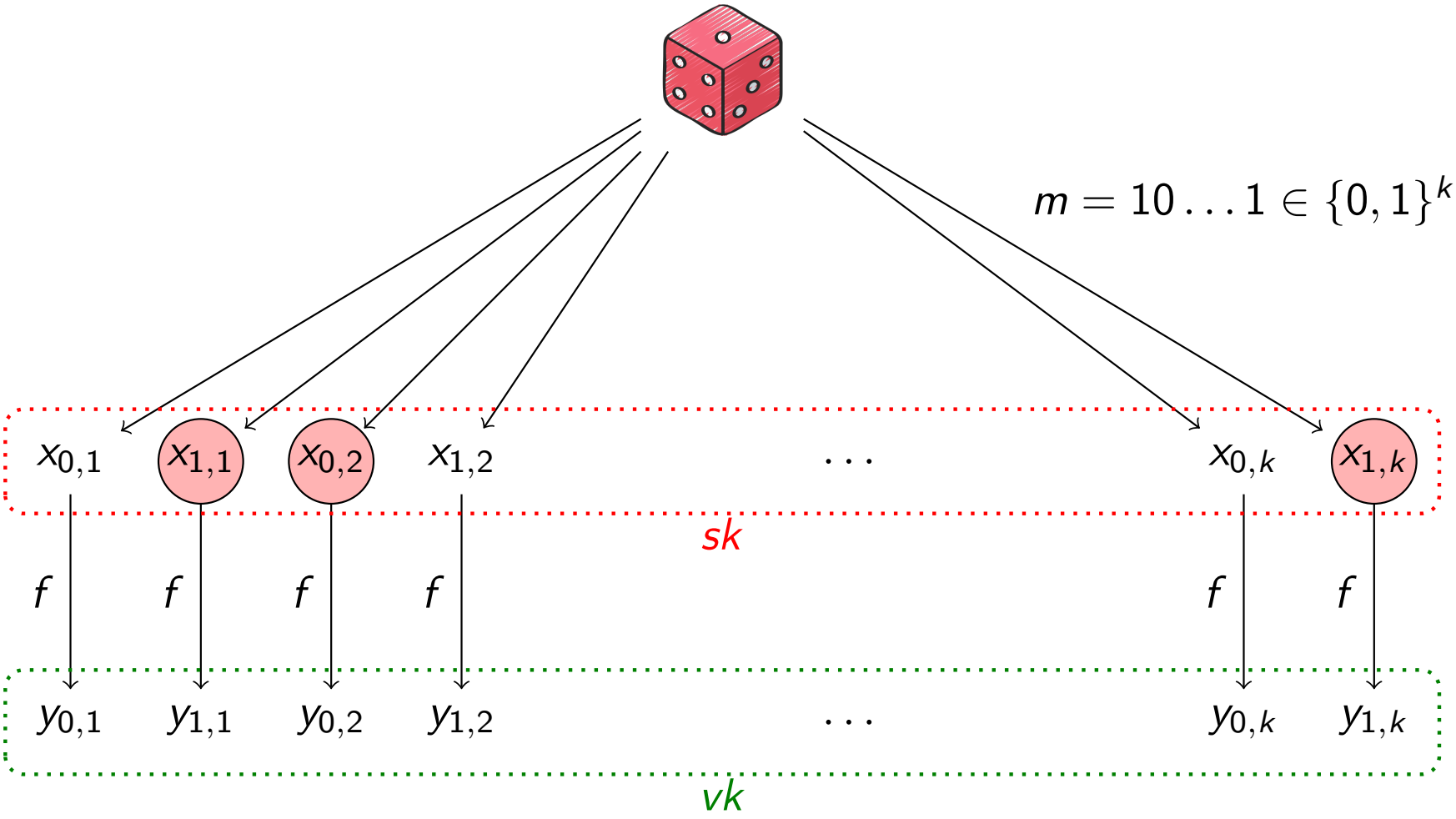
  Verification key $vk = (y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$ which can be published.
  Signing key $sk = (x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ which needs to be kept secret

- **Signature:** if Alice wants to sign $m = m_1 \ldots m_k$, she does the following:
  - Use her signing key $(x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ to send the signature $s = (x_{m_1,1}, x_{m_1,2}, \ldots, x_{m_k,k})$ to Bob.
- **Verification:** to check the validity of $s = (s_1, \ldots, s_k)$ on m, Bob does the following:
  - Obtain Alice's authentic verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$.
  - Check whether $f(s_i) = y_{m_b,i}$ for all $i \in [\![1,k]\!]$.

How to sign k bits **just once** ?



How to sign k bits **just once** ?



How to sign k bits **just once** ?

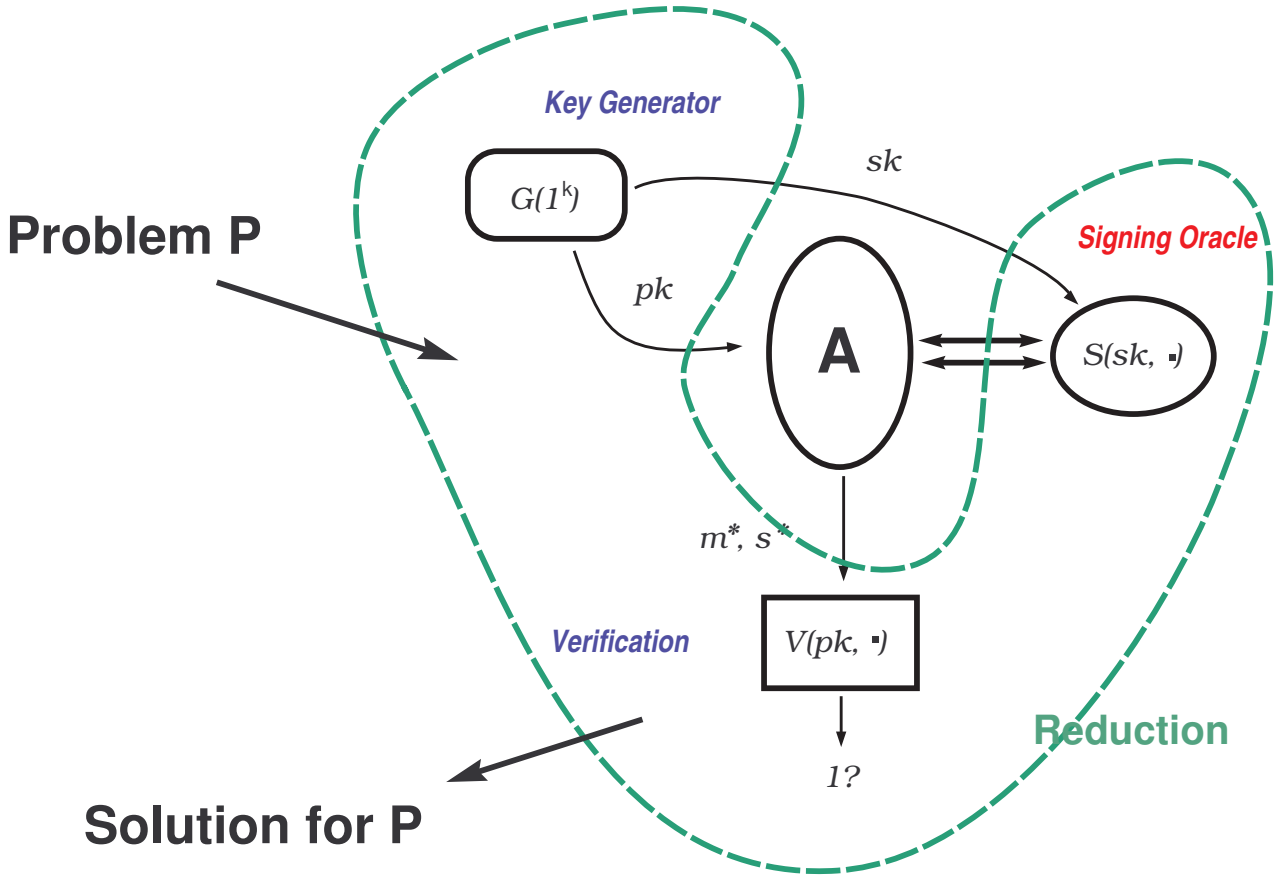
Theorem

If f is (τ, ε)-one way then Lamport's signature scheme (for k-bit messages) is $(1, \tau', 2k \cdot \varepsilon)$-EUF-CMA secure, with $\tau' = \tau + (2k-1)\mathcal{T}_{\mathsf{Eval}}$.

- In other words: If there is an Adversary $\mathcal{A}$ that chooses
  - a message $m \in \{0,1\}^k$ for Alice to legitimately authenticate
  - forges a message $m' \neq m$ with probability at least ε

  Then there is an Adversary $\mathcal{B}$ that can break the one-wayness of the function f with probability at least $\varepsilon/2k$ operates in time roughly the same as $\mathcal{A}$

Simulating the Attacker's Environment



How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

Proof. $\mathcal{B}$ gets as input the description of f and $y^\star \in Y$.

- $\mathcal{B}$ picks as input an index $(i^\star, j^\star) \in \{0,1\} \times [\![1,k]\!]$
- $\mathcal{B}$ selects $2k-1$ random elements $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k} \in X$.
- $\mathcal{B}$ computes their images $y_{i,j} = f(x_{i,j}) = \mathsf{Eval}(x_{i,j})$ for $(i,j) \in \{0,1\} \times [\![1,k]\!] \setminus \{(i^\star, j^\star)\}$.
- $\mathcal{B}$ sets $y_{i^\star,j^\star} = y^\star$
- $\mathcal{B}$ executes $\mathcal{A}$ on the verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$
- At some point $\mathcal{A}$ query **one** message $m = m_1 \ldots m_k$ to the signature oracle
  - If $m_{j^\star} = i^\star$ then $\mathcal{B}$ aborts the simulation (probability $1/2$),
  - otherwise $\mathcal{B}$ outputs a valid signature on m thanks to its knowledge of $x_{0,1}, \ldots, \widehat{x_{i^\star,j^\star}}, \ldots, x_{1,k}$.
- Eventually, $\mathcal{A}$ outputs a signature s' on a message $m' \neq m$ and $\mathcal{B}$ outputs $s'_{j^\star}$. The message m' differs from m in at least one position. If it is the $j^\star$-th position (probability $1/k$) and if the signature is valid (probability ε) we have $f(s'_{j^\star}) = y_{i^\star,j^\star} = y^\star$.

$\square$

How to sign k bits **just once** ?

- Lamport's scheme is EUF-CMA secure assuming **only** the one-wayness of f.
- The signature generation is very efficient.

How to sign k bits **just once** ?

- Lamport's scheme is EUF-CMA secure assuming **only** the one-wayness of f.
- The signature generation is very efficient.

- For (generic) groups of of prime order q of n-bits, solving the discrete logarithm problem requires $2^{n/2}$ operations.
- For a 128-bit security level, we need to have a group order q of (at least) 256 bits and for an ideal $\mathbb{G}$ (an elliptic curve?), elements in $\mathbb{G}$ can be represented with 256 bits.
  The verification key is made of $256^2 = 65536$ bits and its generation requires 256 exponentiation in $\mathbb{G}$.
- The signature is made of k elements from $\mathbb{Z}_q$. The signature length is at least $256 \cdot k$ bits.
- Can sign only one message

Outline

1. Introduction
   - Definitions
   - Security Notions for Digital Signatures
2. Lamport signatures
   - Signing one bit
   - Signing k bits
3. Lamport signatures - Variants
   - Signing $\leq k$ bits
   - Short secret/verification keys
4. Groth's one-time signatures
5. SPHINCS
   - Optimized Lamport signatures
   - Winternitz signatures
   - HORS signatures
   - SPHINCS Signatures

How to sign $\leq$ k bits **just once** ?

Is it OK to sign any message of length $\leq k$?

- **Key generation:**
  - Generate $f : X \longrightarrow Y$ a **one-way function**.
  - Select $2k$ random elements $x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k} \in X$.
  - Compute their images $y_{i,j} = f(x_{i,j}) = \mathrm{Eval}(x_{i,j})$ for $i \in \{0,1\}$ and $j \in [\![1,k]\!]$.

  Public key $= (y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$ which can be published.
  Private key $= (x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ which needs to be kept secret

- **Signature:** if Alice wants to sign $m = m_1 \ldots m_\ell$, she does the following:
  - Use her private key $(x_{0,1}, x_{1,1}, \ldots, x_{0,k}, x_{1,k})$ to send the signature $s = (x_{m_1,1}, x_{m_1,2}, \ldots, x_{m_\ell,\ell})$ to Bob.
- **Verification:** to check the validity of $s = (s_1, \ldots, s_\ell)$ on m, Bob does the following:
  - Obtain Alice's authentic verification key $(y_{0,1}, y_{1,1}, \ldots, y_{0,k}, y_{1,k})$.
  - Check whether $f(s_i) = y_{m_b,i}$ for all $i \in [\![1,\ell]\!]$.

An example

Dear Sir/Madam,

while working with Charles, I have always found him working studiously and sincerely at his table without gossiping with colleagues in the office. He seldom wastes his time on useless things. Given a job, he always finishes the given assignment in time. He is always deeply engrossed in his official work, and can never be found chitchatting in the canteen. He has absolutely no vanity in spite of his high accomplishment and profound knowledge of his field. I think he can easily be classed as outstanding, and should on no account be dispensed with. I strongly feel that Charles should be pushed to accept promotion, and a proposal to management be sent away as soon as possible.

Best regards,

Damien

An example

Dear Sir/Madam,

while working with Charles, I have always found him
working studiously and sincerely at his table without
gossiping with colleagues in the office. He seldom
wastes his time on useless things. Given a job, he always
finishes the given assignment in time. He is always
deeply engrossed in his official work, and can never be
found chitchatting in the canteen. He has absolutely no
vanity in spite of his high accomplishment and profound
knowledge of his field. I think he can easily be
classed as outstanding, and should on no account be
dispensed with. I strongly feel that Charles should be
pushed to accept promotion, and a proposal to management be
sent away as soon as possible.

Best regards,

Damien

An example

Dear Sir/Madam,

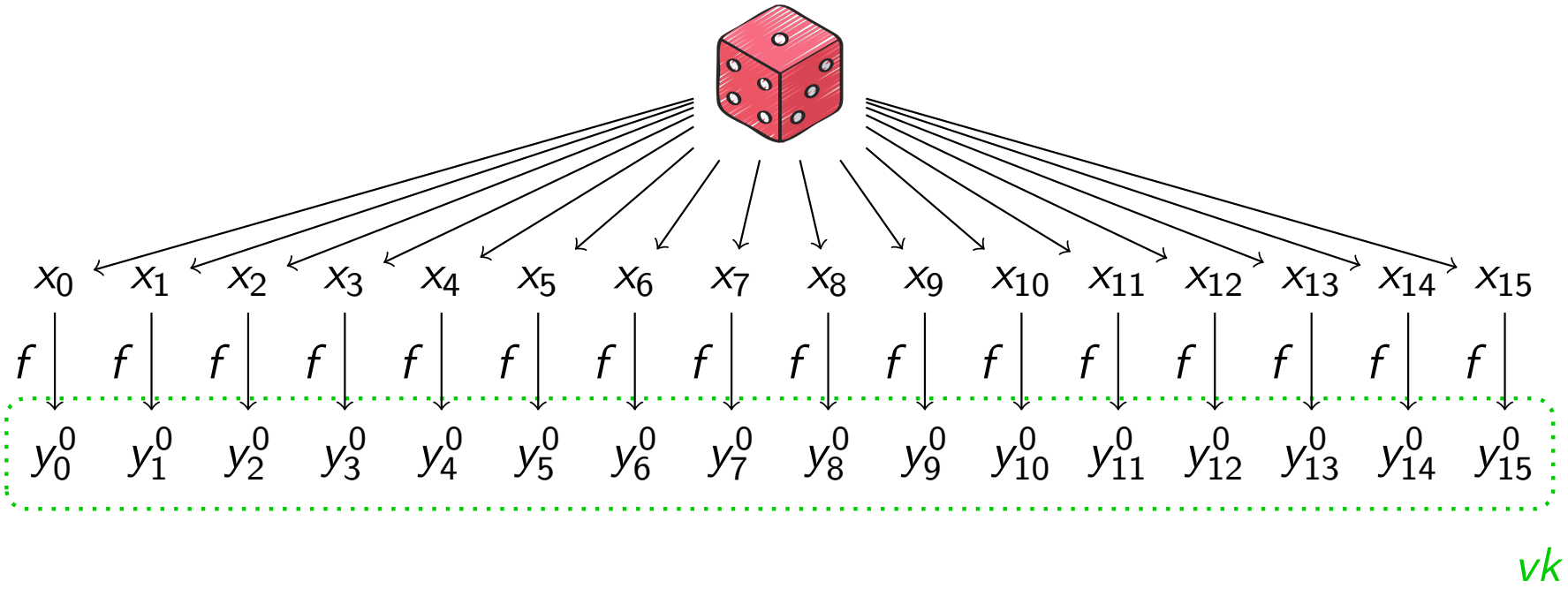
while working with Charles, I have always found him
working studiously and sincerely at his table without
gossiping with colleagues in the office. He seldom
wastes his time on useless things. Given a job, he always
finishes the given assignment in time. He is always
deeply engrossed in his official work, and can never be
found chitchatting in the canteen. He has absolutely no
vanity in spite of his high accomplishment and profound
knowledge of his field. I think he can easily be
classed as outstanding, and should on no account be
dispensed with. I strongly feel that Charles should be
pushed to accept promotion, and a proposal to management be
sent away as soon as possible.

Best regards,

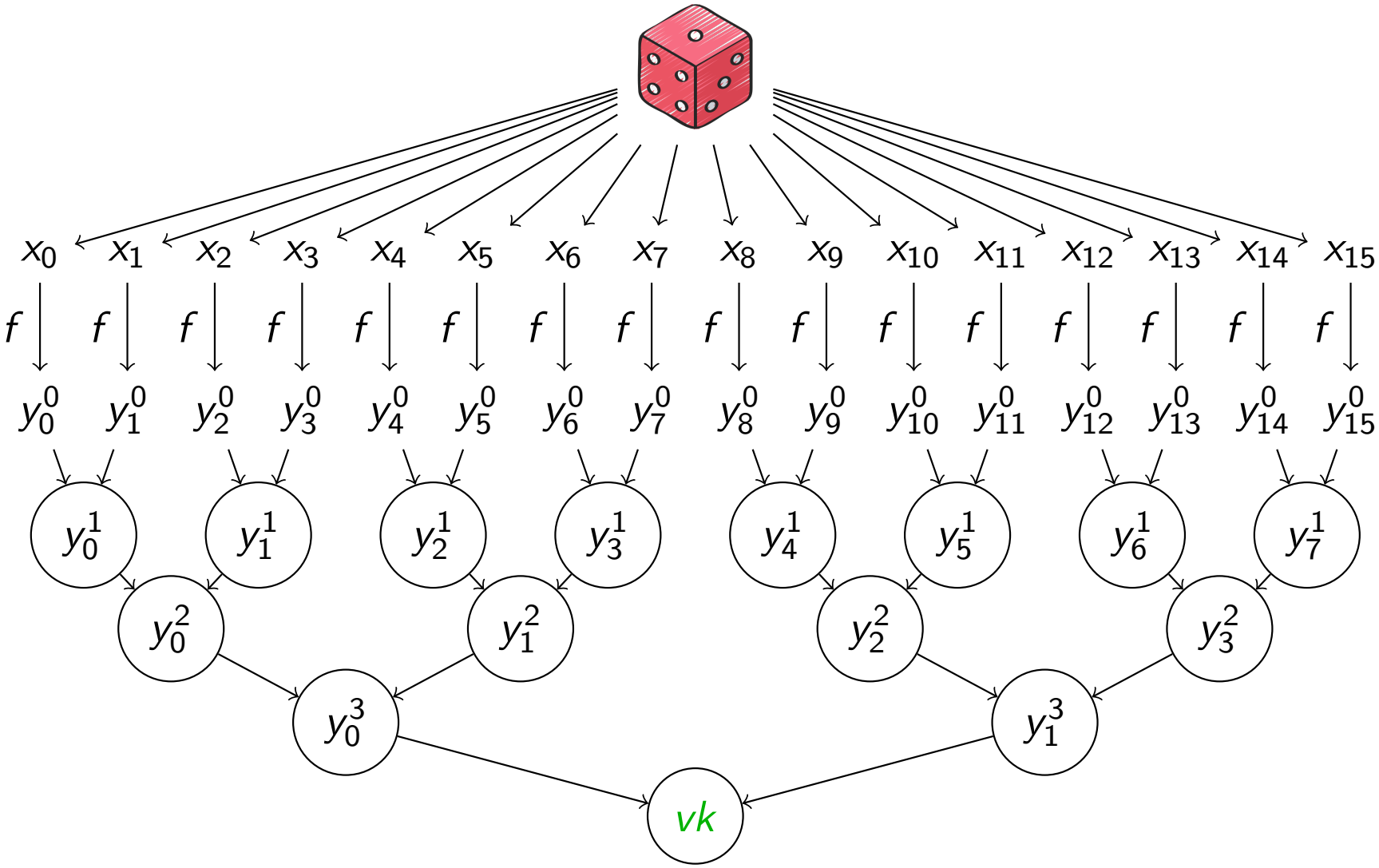
Damien

$\rightsquigarrow$ see **TD** for a secure variant

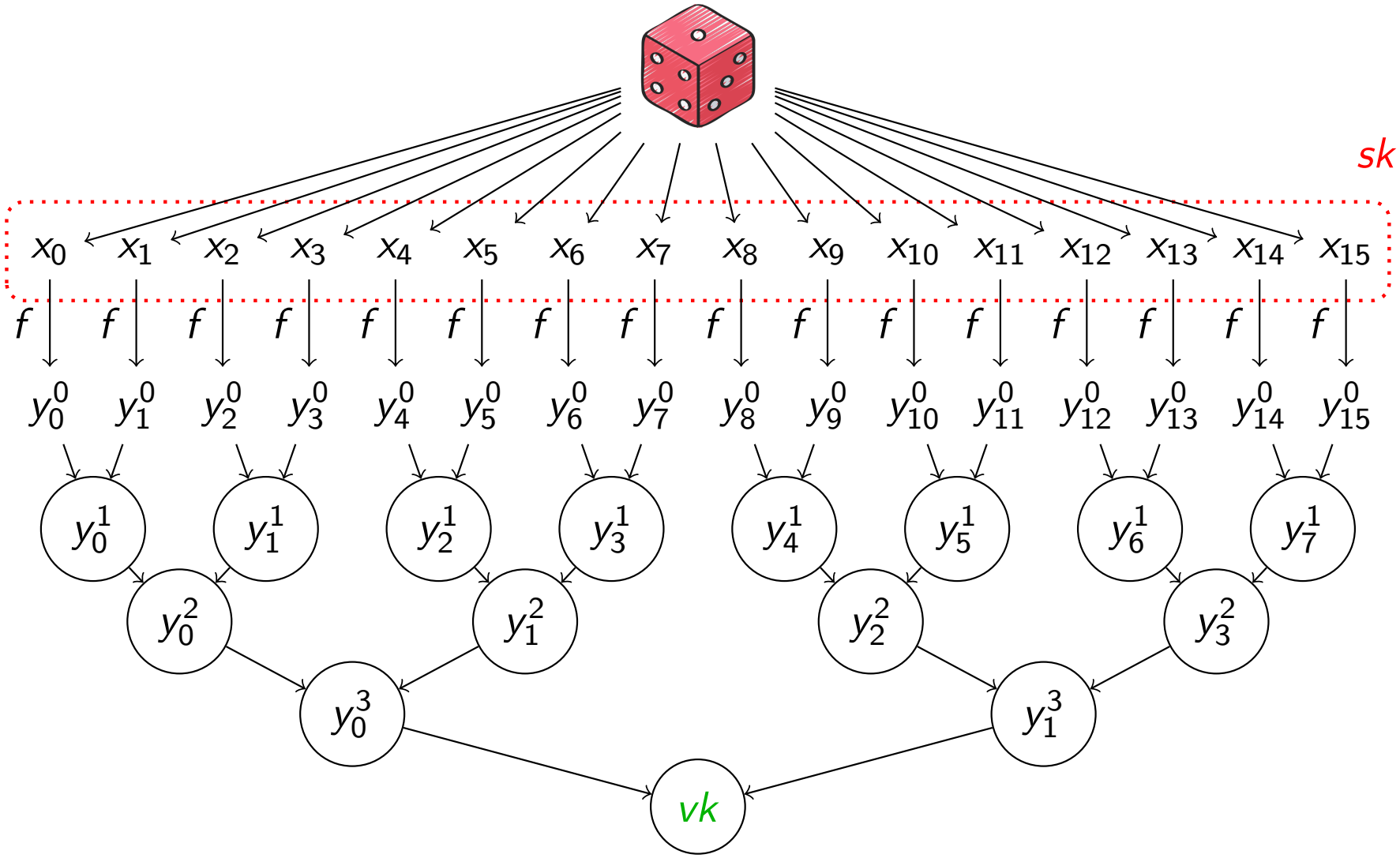
Short Verification Key – Merkle Hash Tree



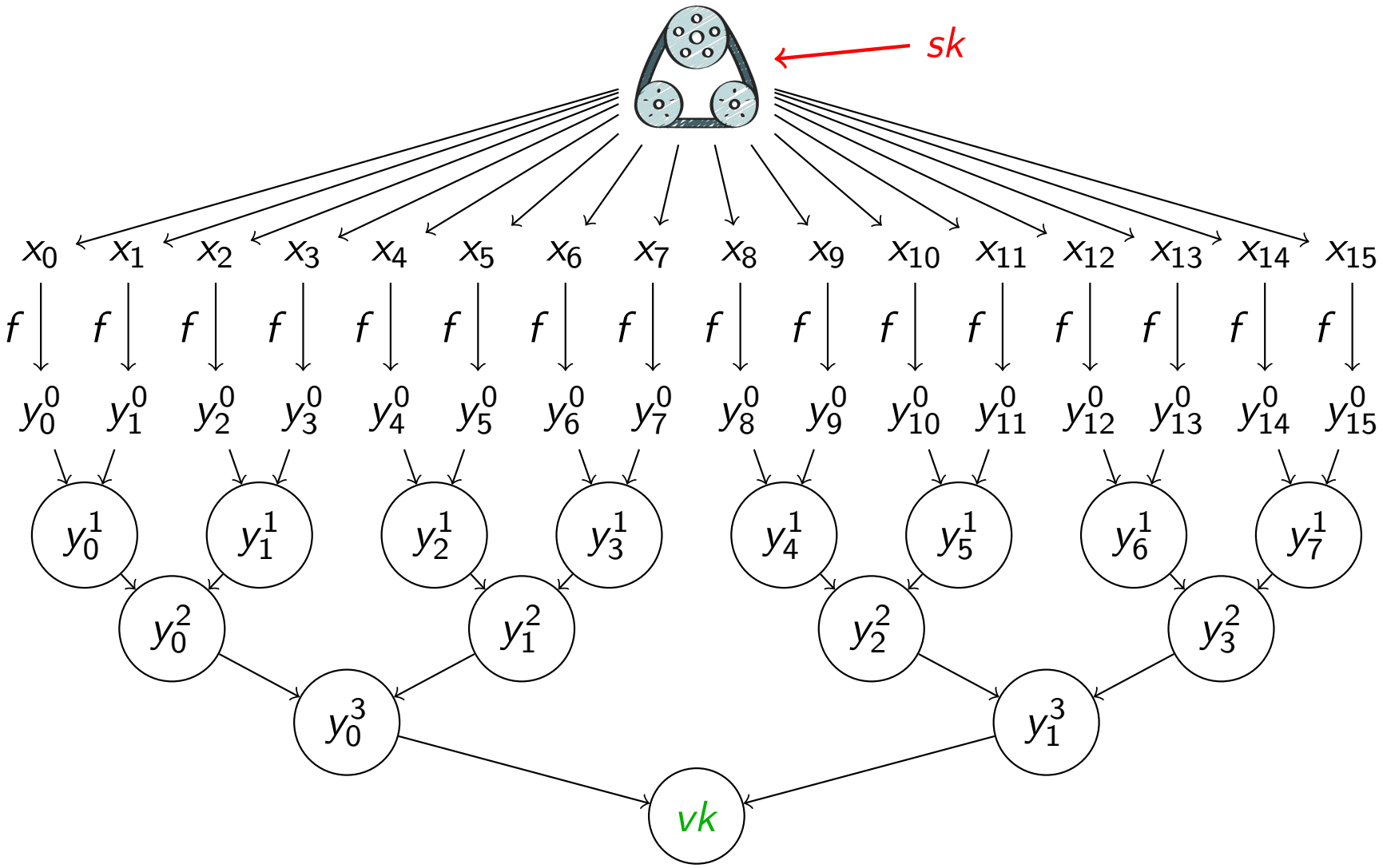
Short Verification Key – Merkle Hash Tree



Short Private Key – Pseudorandom generator

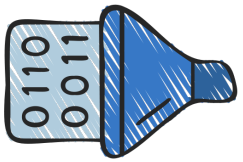


Short Private Key – Pseudorandom generator



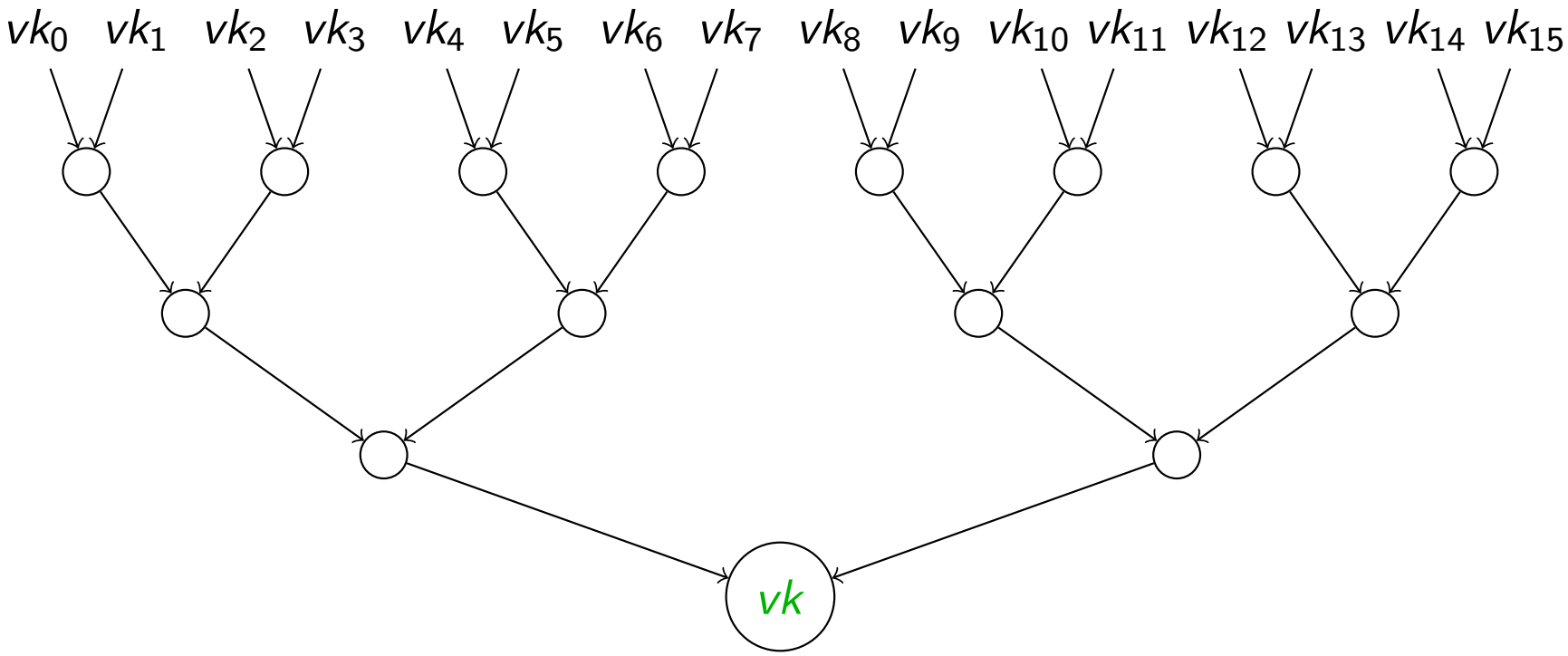
Hashing the message

- Lamport signatures **do not** require hashing the message m before signing it **but** can only sign messages of length k
- $\rightsquigarrow$ use a collision-resistant hash function H and sign $H(m)$ instead of m.

$$m \in \{0,1\}^* \longrightarrow$$  $$\longrightarrow H(m) \in \{0,1\}^k$$

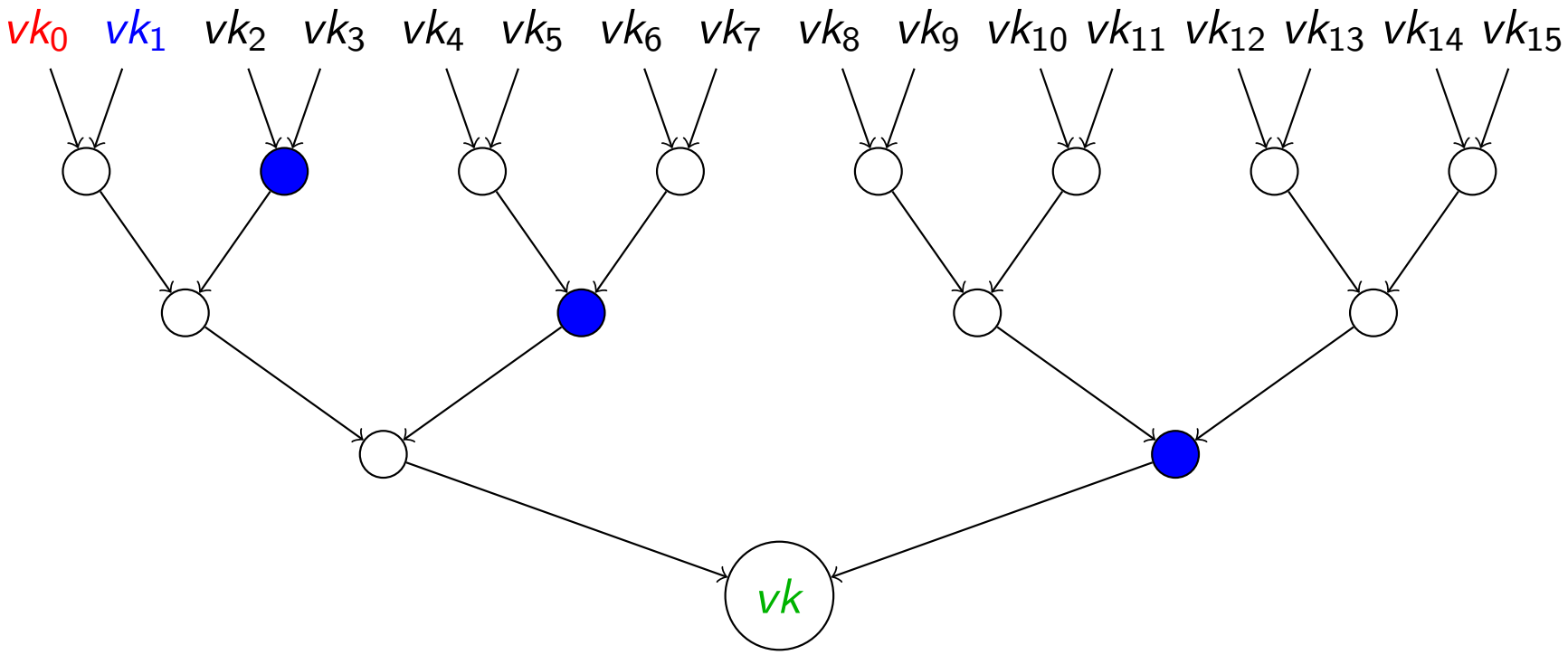
- requires collision-resistance of H ($\rightsquigarrow$ k is twice the security parameter)
- provides better security (see **TD**)

Several messages - Merkle strikes back



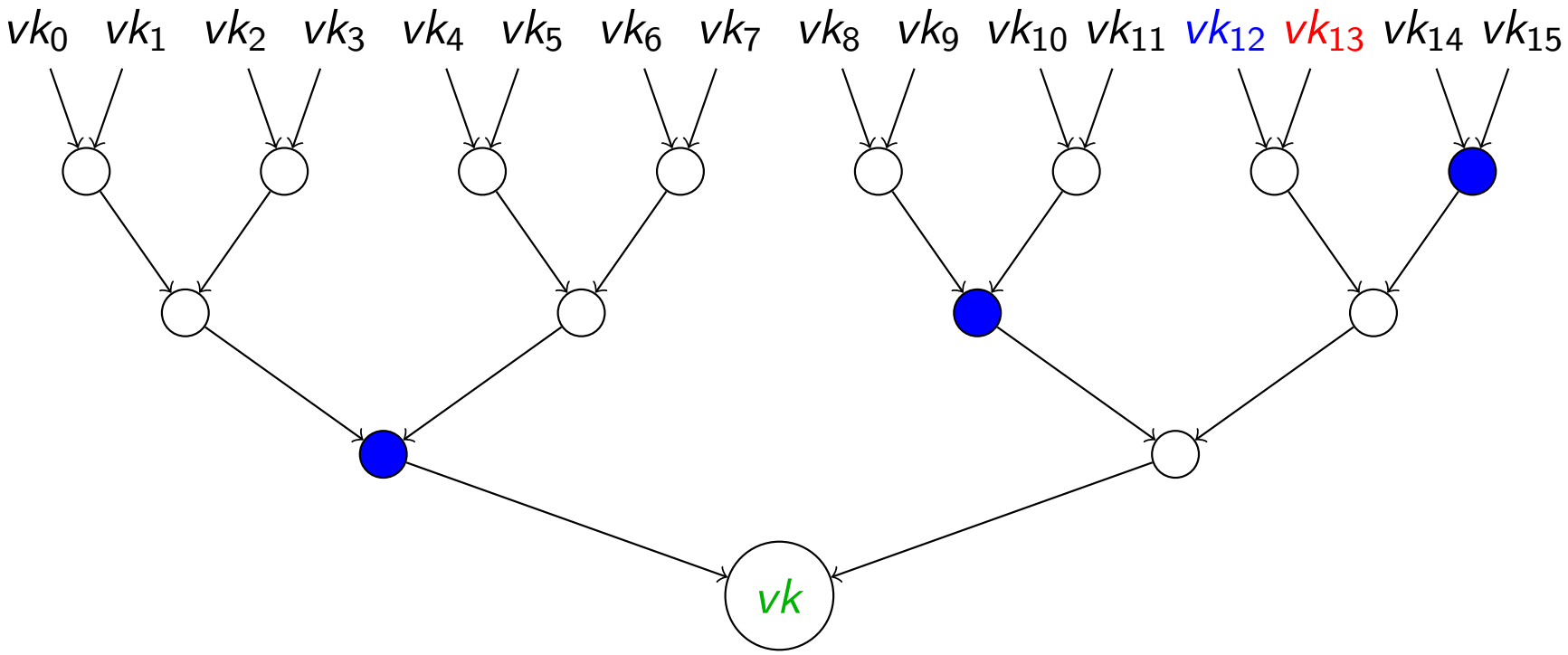
- many keys have to be published if many messages are to be signed.
- a hash tree can be used on those public keys, publishing the top hash of the hash tree instead.
- this increases the size of the resulting signature (parts of the hash tree have to be included in the signature)

Several messages - Merkle strikes back



- many keys have to be published if many messages are to be signed.
- a hash tree can be used on those public keys, publishing the top hash of the hash tree instead.
- this increases the size of the resulting signature (parts of the hash tree have to be included in the signature)

Several messages - Merkle strikes back



- many keys have to be published if many messages are to be signed.
- a hash tree can be used on those public keys, publishing the top hash of the hash tree instead.
- this increases the size of the resulting signature (parts of the hash tree have to be included in the signature)

Outline

1. Introduction
   - Definitions
   - Security Notions for Digital Signatures
2. Lamport signatures
   - Signing one bit
   - Signing k bits
3. Lamport signatures - Variants
   - Signing $\leq k$ bits
   - Short secret/verification keys
4. Groth's one-time signatures
5. SPHINCS
   - Optimized Lamport signatures
   - Winternitz signatures
   - HORS signatures
   - SPHINCS Signatures

Groth's one-time signatures

Groth (2006)
Simulation-sound NIZK proofs for a practical language and constant size group signatures.
Advances in Cryptology - Asiacrypt 2006: pp. 444–459

Key generation: generate $vk = (X = g^x, Y = g^y, Z = g^z)$ where $x, y, z \xleftarrow{\$} \mathbb{Z}_p^*$

Sign: to sign $m \in \mathbb{Z}_p^*$, select $r \xleftarrow{\$} \mathbb{Z}_p^*$, compute $s = (1 - mx - yr)/z \in \mathbb{Z}_p^*$, and output $\sigma = (r, s)$.

Verify: given $\sigma \in (\mathbb{Z}_p^*)^2$, check

$$X^m Y^r Z^s = g.$$

Groth's one-time signatures

Theorem

If the discrete logarithm assumption holds in $\mathbb{G}$ then Groth's signature scheme is one-time EUF-CMA secure.

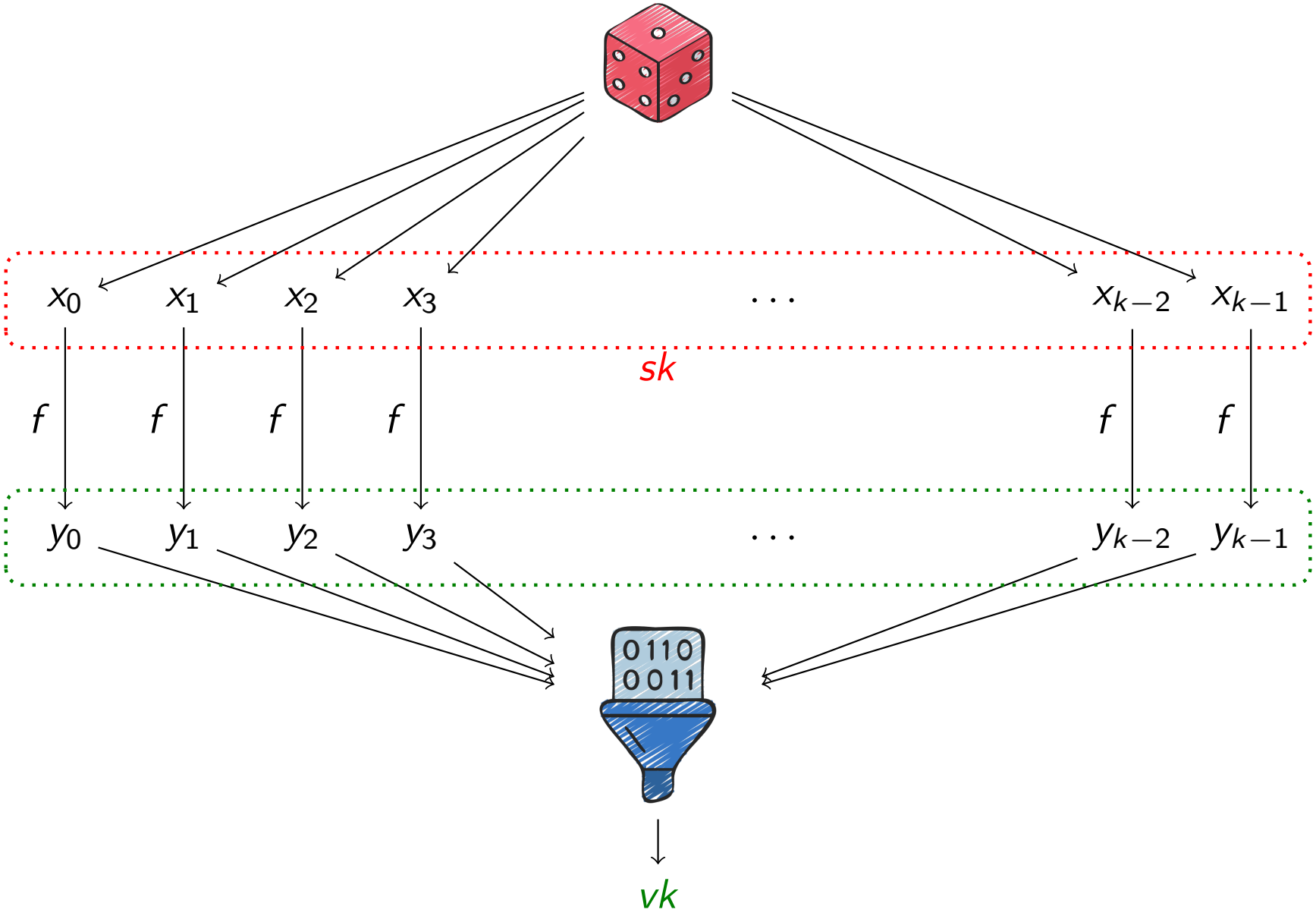
Proof idea: given a DL instance $(g, h) \in \mathbb{G}$, one sets $X = g^{a_1} h^{b_1}$, $Y = g^{a_2} h^{b_2}$, $Z = g^{a_3}$ where $a, b, c \xleftarrow{\$} \mathbb{Z}_p^*$. On signature query on m, one compute $r = -mb_1/b_2 \bmod p$ and $s = (1 - ma_1 - r_2)/a_3 \bmod p$.

Thanks to the adversary's forgery, one can retrieve the discrete logarithm of h in base g by solving a linear system. $\square$

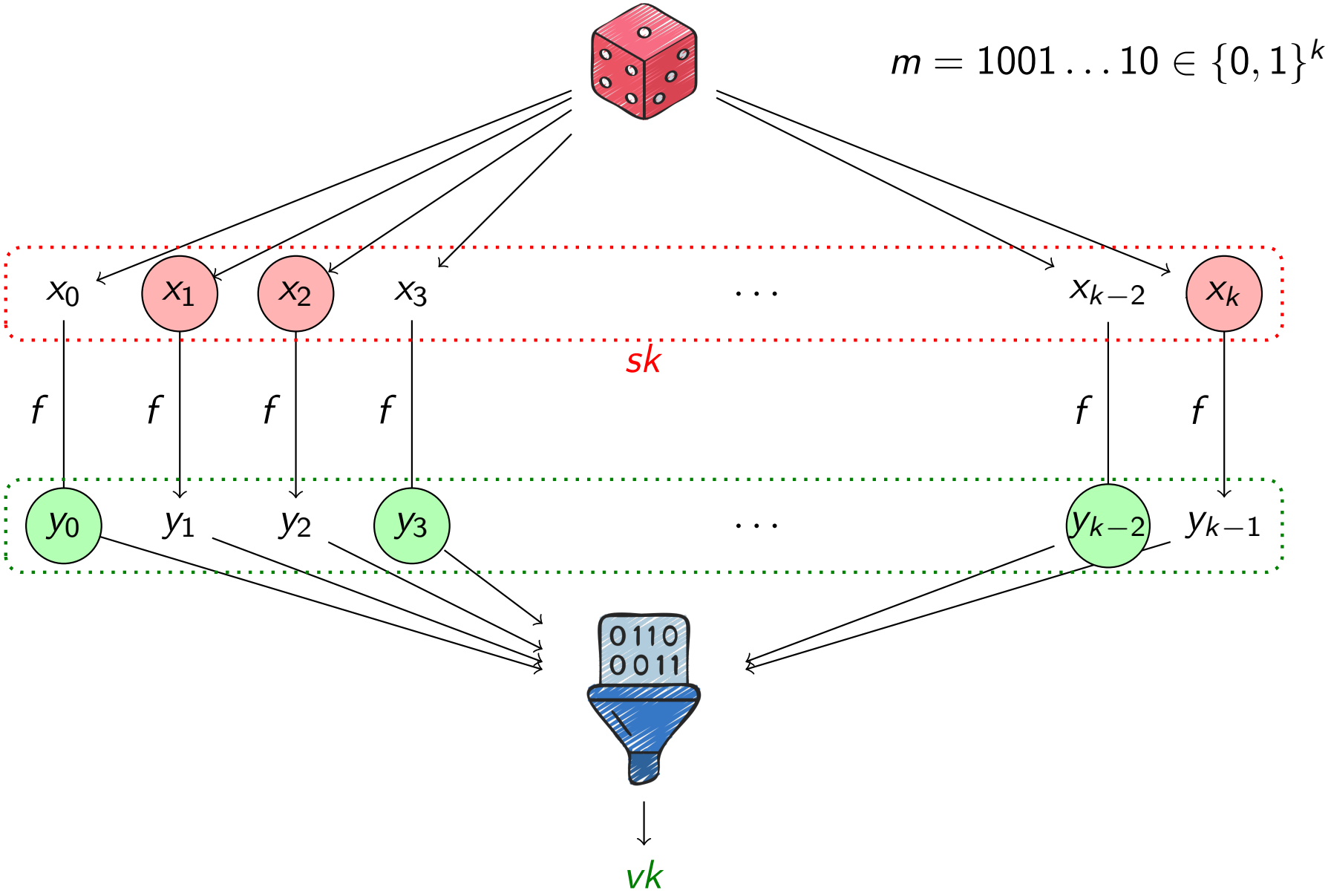
Outline

1. Introduction
   - Definitions
   - Security Notions for Digital Signatures
2. Lamport signatures
   - Signing one bit
   - Signing k bits
3. Lamport signatures - Variants
   - Signing $\leq k$ bits
   - Short secret/verification keys
4. Groth's one-time signatures
5. SPHINCS
   - Optimized Lamport signatures
   - Winternitz signatures
   - HORS signatures
   - SPHINCS Signatures

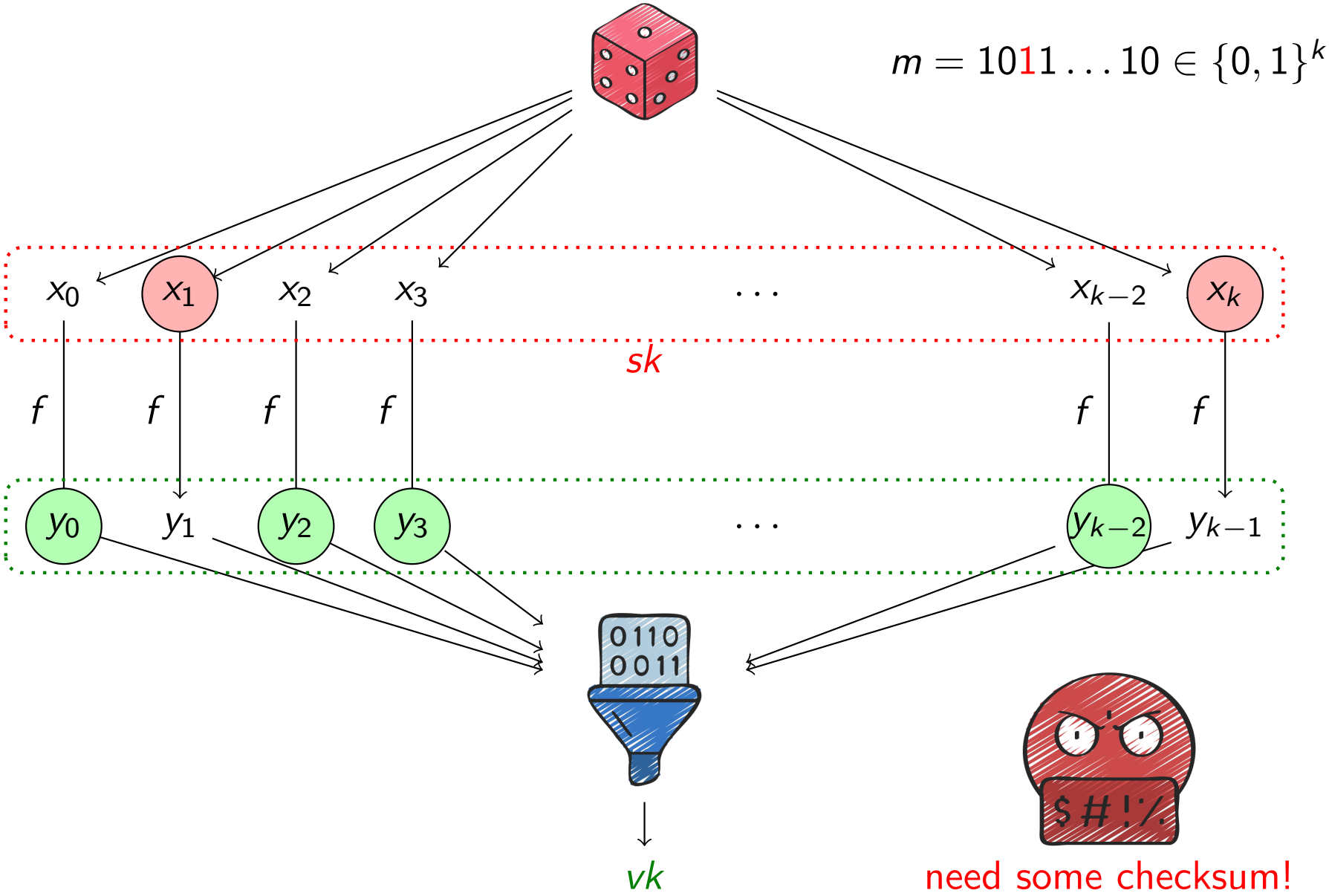
Optimized Lamport signatures



Optimized Lamport signatures



Optimized Lamport signatures



Optimized Lamport signatures

$\mathcal{M} = \{0,1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log(k) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in [\![0, \ell]\!]$.

  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log(k) \rceil}}(x_\ell)).$$

- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - check whether

$$vk = H((f^{1-m_0}(s_0), \ldots, f^{1-m_{k-1}}(s_{k-1}), f^{1-c_0}(s_k), \ldots, f^{1-c_{\lceil \log(k) \rceil}}(s_\ell)))$$

Optimized Lamport signatures

$\mathcal{M} = \{0,1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log(k) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in [\![0, \ell]\!]$.

  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log(k) \rceil}}(x_\ell)).$$

- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - check whether

$$vk = H((f^{1-m_0}(s_0), \ldots, f^{1-m_{k-1}}(s_{k-1}), f^{1-c_0}(s_k), \ldots, f^{1-c_{\lceil \log(k) \rceil}}(s_\ell)))$$

Optimized Lamport signatures

$\mathcal{M} = \{0,1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log(k) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f(x_i)$ for $i \in [\![0, \ell]\!]$.

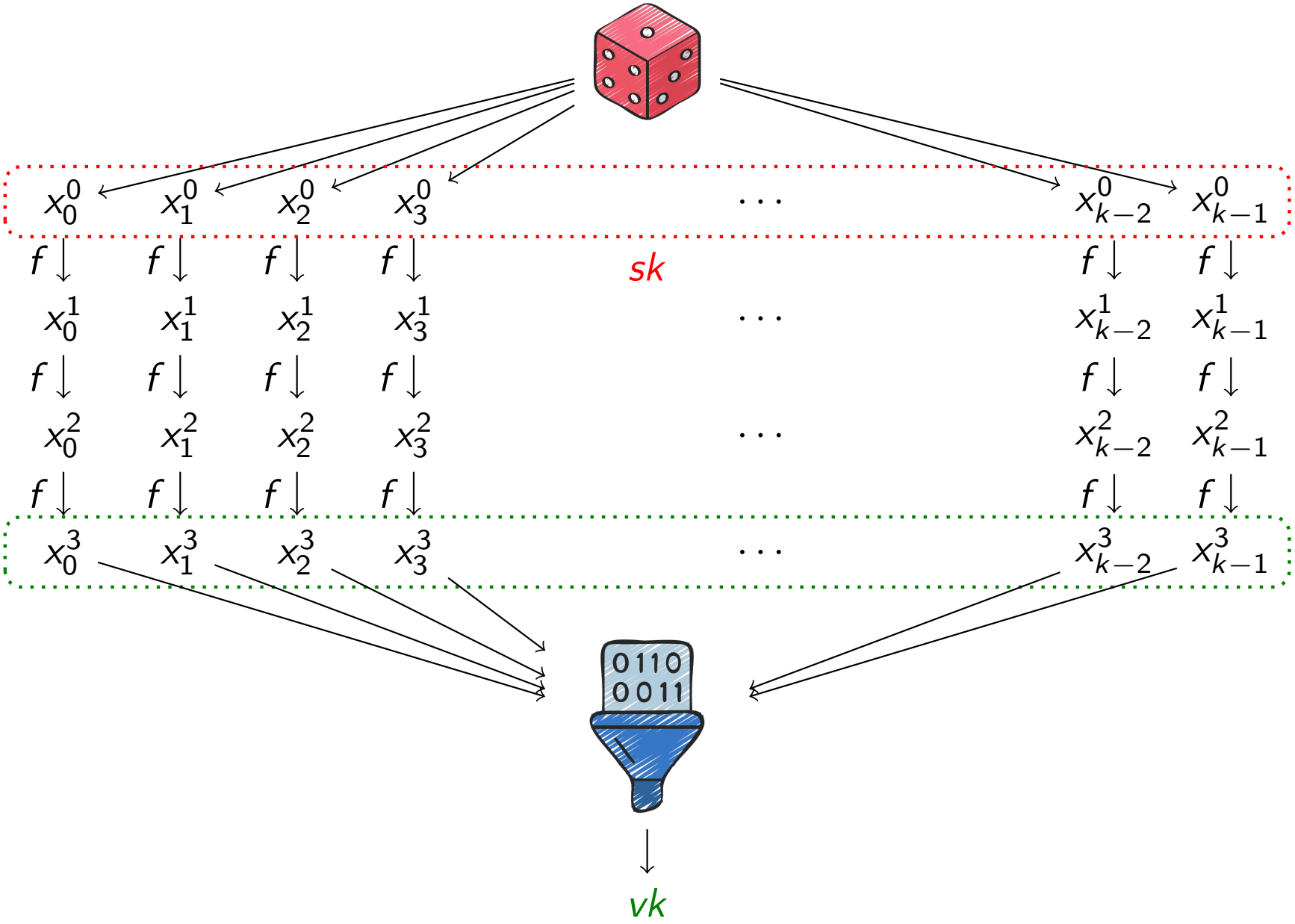
  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log(k) \rceil}}(x_\ell)).$$

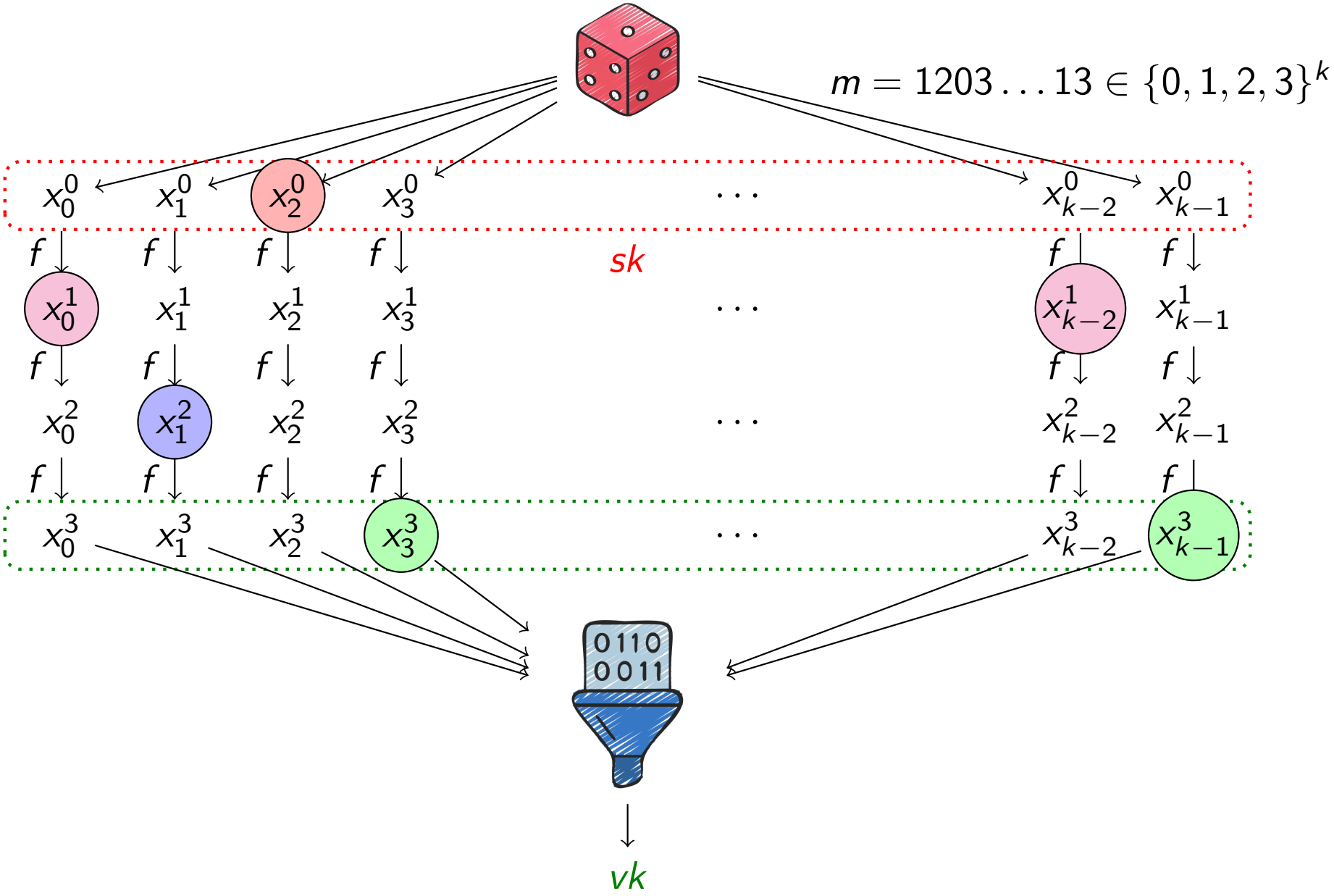
- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - compute the checksum $c = k - \sum_{i=0}^{k-1} m_i$ and write it in binary $c = (c_0, \ldots, c_{\lceil \log(k) \rceil})$
  - check whether

$$vk = H((f^{1-m_0}(s_0), \ldots, f^{1-m_{k-1}}(s_{k-1}), f^{1-c_0}(s_k), \ldots, f^{1-c_{\lceil \log(k) \rceil}}(s_\ell)))$$

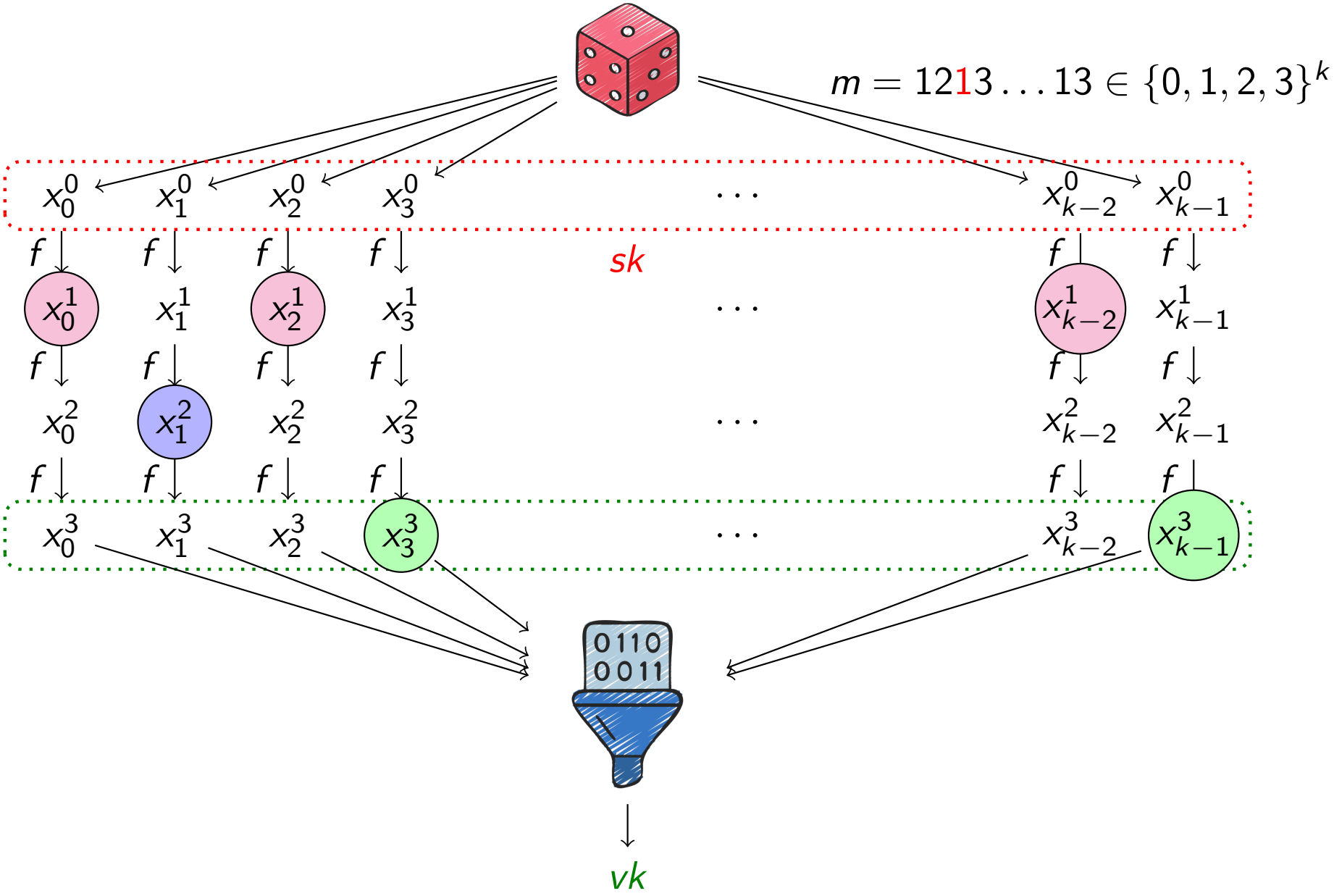
Winternitz signatures



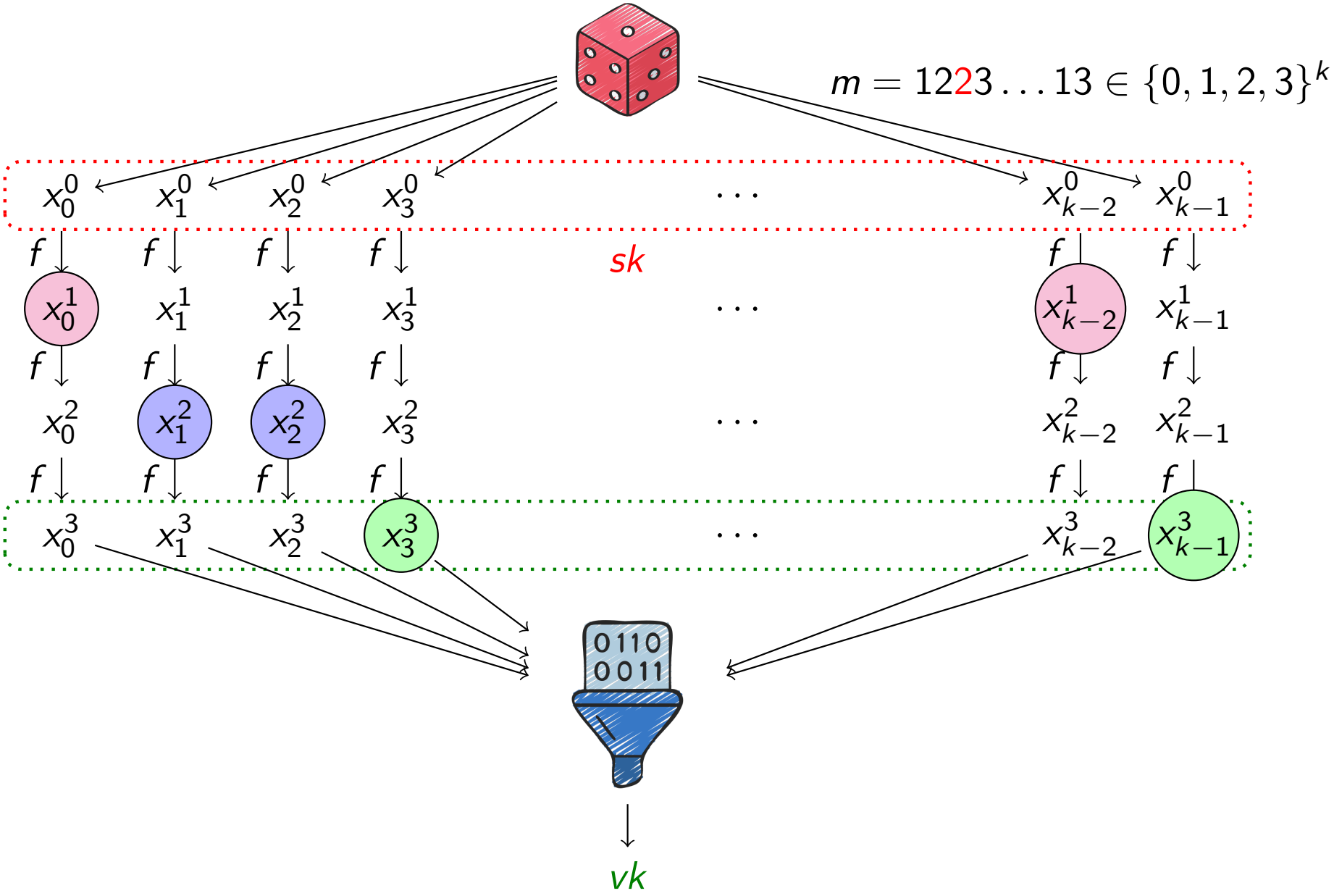
Winternitz signatures



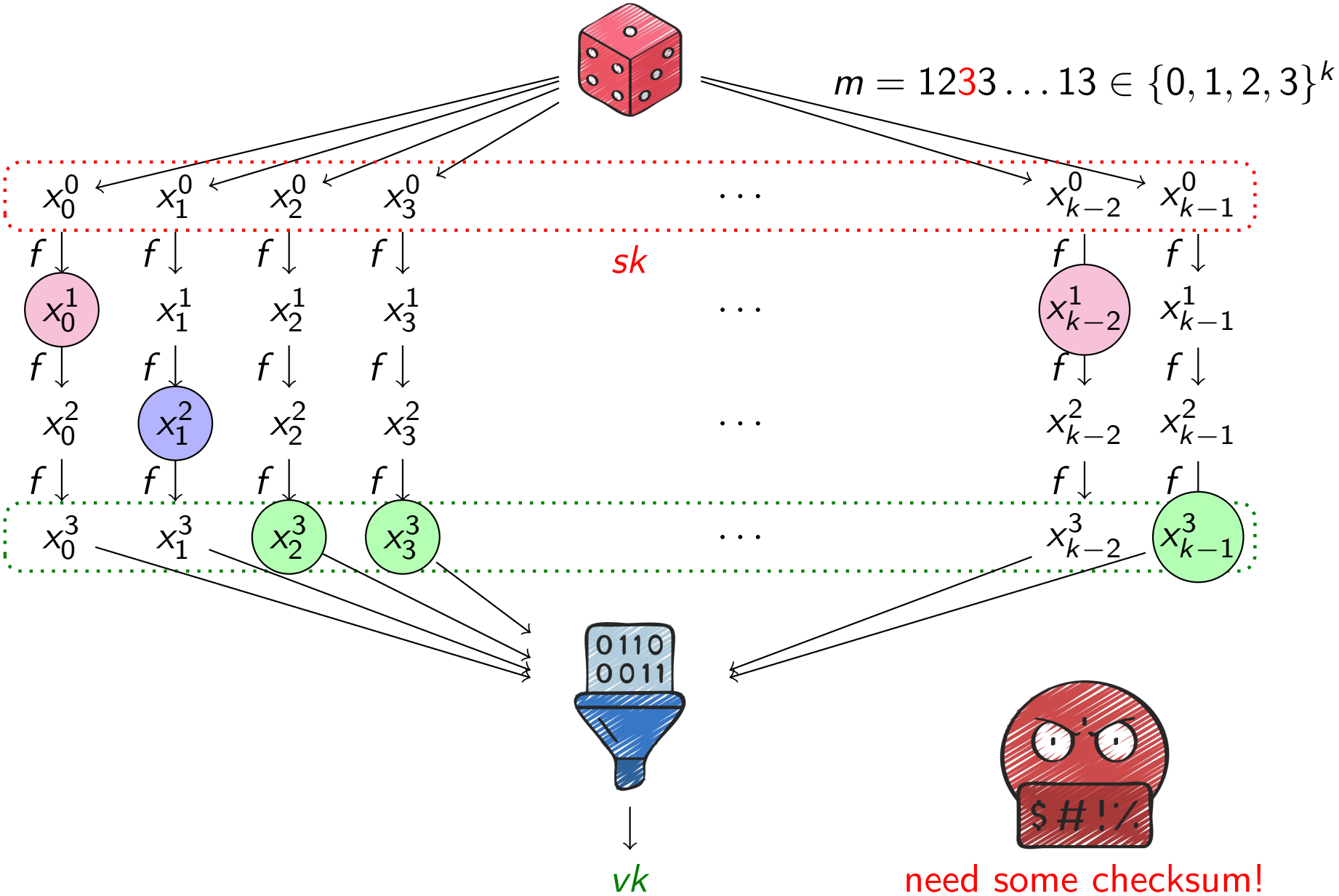
Winternitz signatures



Winternitz signatures



Winternitz signatures



need some checksum!

Winternitz signatures

$\mathcal{M} = \{0, 1, 2, \ldots, t-1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log_t(kt) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f^{t-1}(x_i)$ for $i \in [\![0, \ell]\!]$.

  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log_t(kt) \rceil}}(x_\ell)).$$

- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - check whether

$$vk = H((f^{t-m_0}(s_0), \ldots, f^{t-m_{k-1}}(s_{k-1}), f^{t-c_0}(s_k), \ldots, f^{t-c_{\lceil \log_t(kt) \rceil}}(s_\ell)))$$

Winternitz signatures

$\mathcal{M} = \{0, 1, 2, \ldots, t-1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log_t(kt) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f^{t-1}(x_i)$ for $i \in [\![0, \ell]\!]$.

  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log_t(kt) \rceil}}(x_\ell)).$$

- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - check whether

$$vk = H((f^{t-m_0}(s_0), \ldots, f^{t-m_{k-1}}(s_{k-1}), f^{t-c_0}(s_k), \ldots, f^{t-c_{\lceil \log_t(kt) \rceil}}(s_\ell)))$$

Winternitz signatures

$\mathcal{M} = \{0, 1, 2, \ldots, t-1\}^k$; *H is a cryptographic hash function*

- **Key generation:**
  - Select $\ell + 1 = k + \lceil \log_t(kt) \rceil + 1$ random elements $x_0, x_1, \ldots, x_{\ell-1}, x_\ell \in X$.
  - Compute their images $y_i = f^{t-1}(x_i)$ for $i \in [\![0, \ell]\!]$.

  $vk = H(y_0, y_1, \ldots, y_{\ell-1}, y_\ell)$; $sk = (x_0, x_1, \ldots, x_{\ell-1}, x_\ell)$
- **Signature:** if Alice wants to sign $m = m_0 \ldots m_{k-1}$, she does the following:
  - compute the checksum $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - use her signing key sk to send to Bob the signature:

$$s = (f^{m_0}(x_0), \ldots, f^{m_{k-1}}(x_{k-1}), f^{c_0}(x_k), \ldots, f^{c_{\lceil \log_t(kt) \rceil}}(x_\ell)).$$

- **Verification:** to check the validity of $s = (s_0, \ldots, s_\ell)$ on m, Bob does the following:
  - $c = k(t-1) - \sum_{i=0}^{k-1} m_i$ and write it in base t: $c = (c_0, \ldots, c_{\lceil \log_t(kt) \rceil})$
  - check whether

$$vk = H((f^{t-m_0}(s_0), \ldots, f^{t-m_{k-1}}(s_{k-1}), f^{t-c_0}(s_k), \ldots, f^{t-c_{\lceil \log_t(kt) \rceil}}(s_\ell)))$$

Winternitz signatures

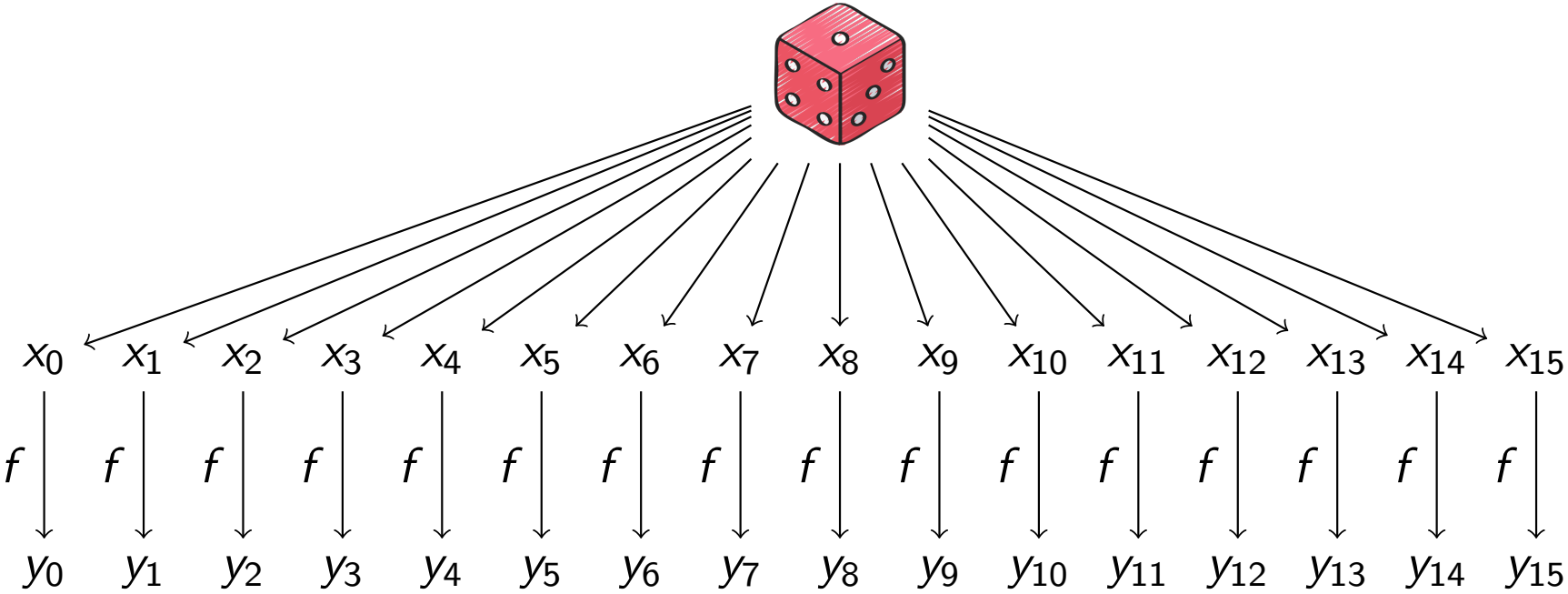
R. C. Merkle
A Certified Digital Signature.
CRYPTO 1989: 218-238
(attributed to R. Winternitz in 1979)

Winternitz one-time signatures (WOTS)

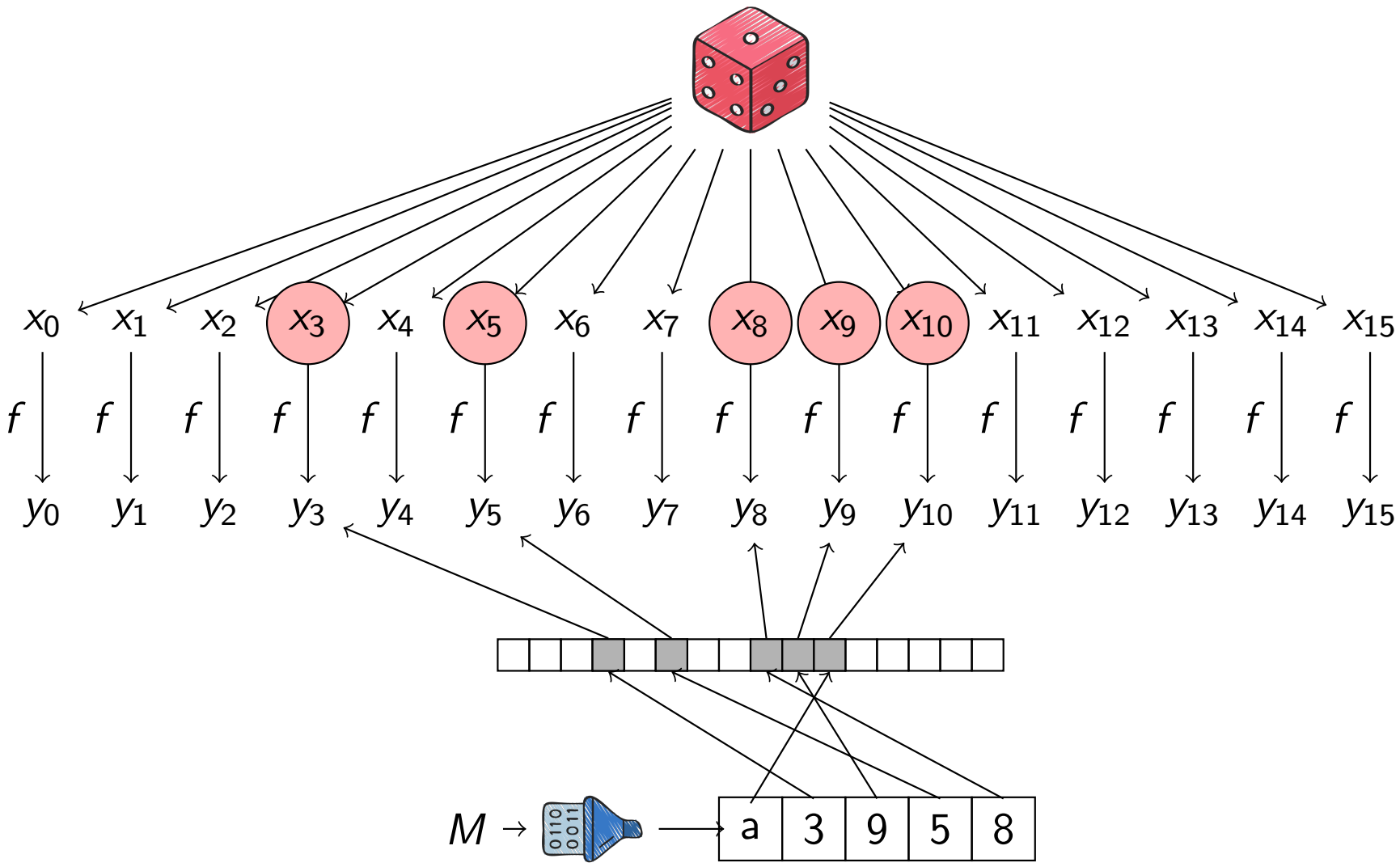
- uses smaller keys
- produces shorter signatures
- generating/verifying signatures is more costly

Exercice: compute the signature/key sizes to sign messages of 256 bits divided into $k = 16$ blocks of integers in $\{0, \ldots, 15\}$ (i.e. $t = 16$) (and compare with Lamport signatures)

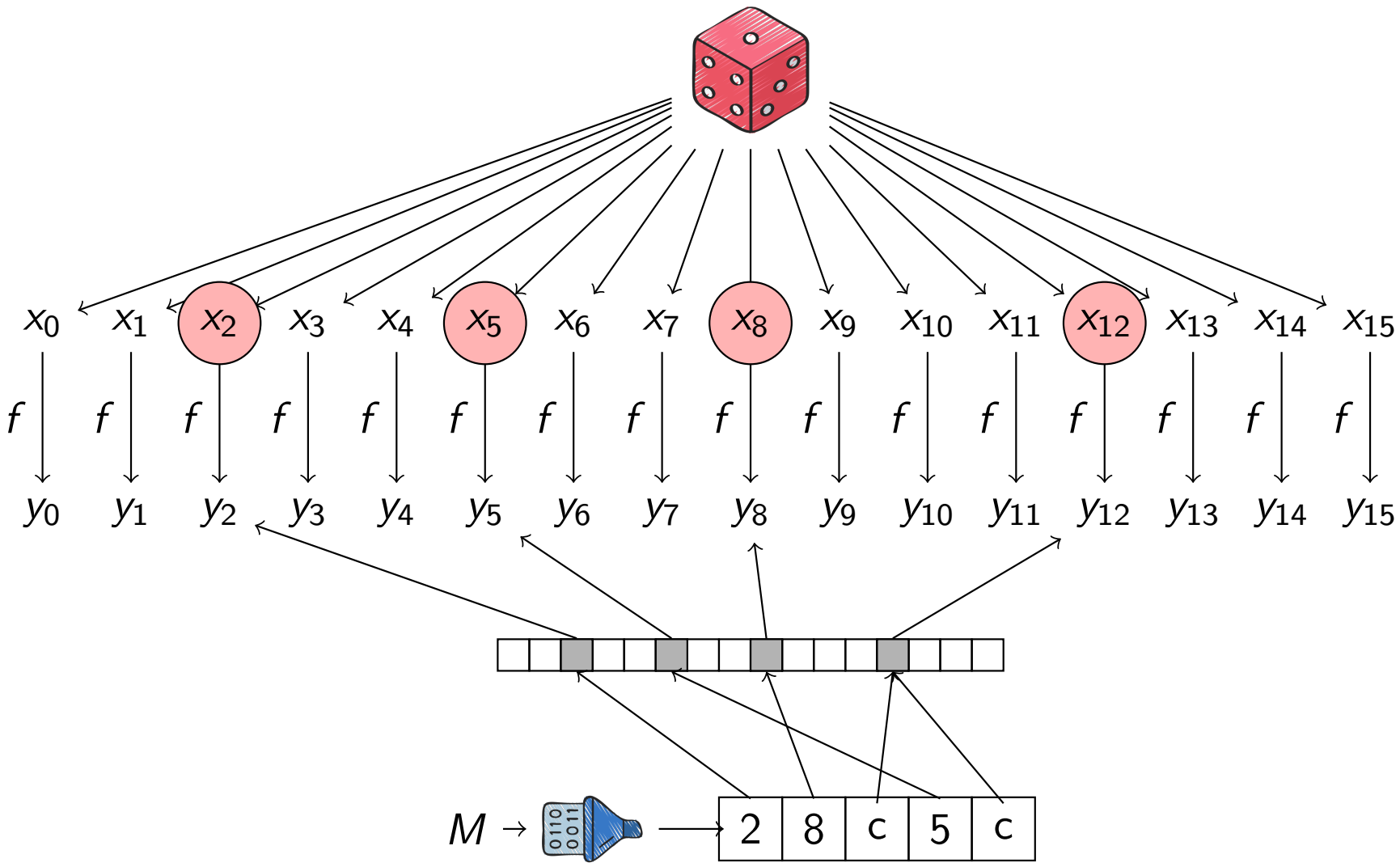
HORS Signatures



HORS Signatures



HORS Signatures



HORS Signatures

L. Reyzin, N. Reyzin
Better than BiBa: Short One-Time Signatures with Fast Signing and Verifying.
ACISP 2002: 144-153

- actually a **few-time signature** scheme
- security relies on the subset-cover resilience of the hash function: Find $m_0, \ldots, m_r$ such that

$$H(m) \subseteq \bigcup_{i=1}^{r} H(m_i)$$

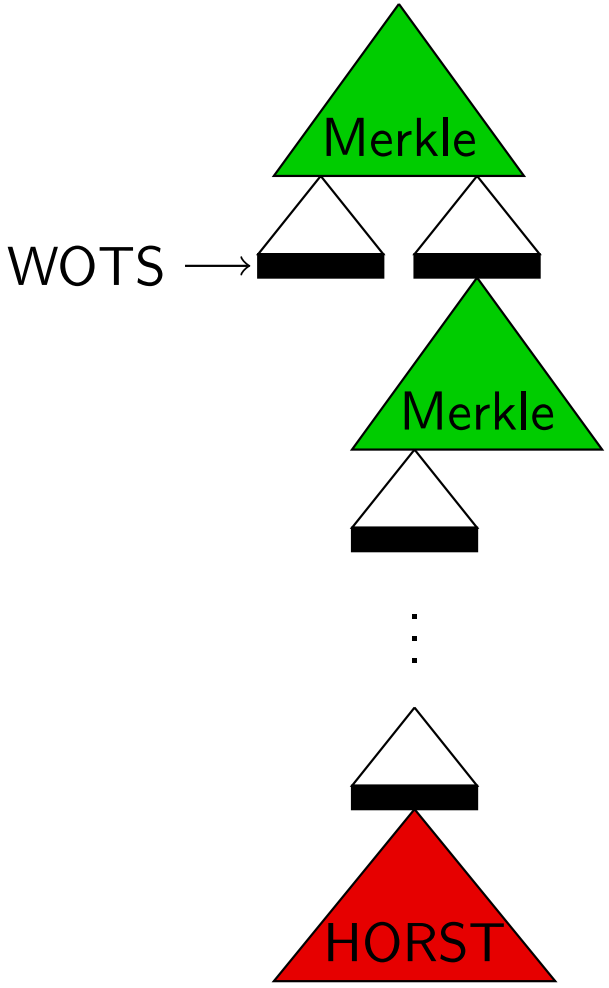
  where r denotes the number of signatures an adversary can query (see **TD**)
- HORST: same idea but use Merkle hash tree to compress the verification key

SPHINCS Signatures

D.I J. Bernstein, D. Hopwood, A. Hülsing, T. Lange, R. Niederhagen, L. Papachristodoulou, M. Schneider, P. Schwabe, Z. Wilcox-O'Hearn
SPHINCS: Practical Stateless Hash-Based Signatures.
EUROCRYPT 2015: 368-397

- stateless many-time signatures
  (classical Merkle tree is **statefull**)
- combines all the techniques we saw before
- a more efficient variant SPHINCS+ was selected in 2022 by the NIST for standardization of a so-called post-quantum signature scheme

SPHINCS Signatures

- (Hyper)tree of WOTS signatures
  ($\simeq$ certificate chain)
  $\rightsquigarrow$ WOTS used to sign the verification key of the next level
- use a PRF of the signed message to determine the leaf for the HORST signature (and the whole path from the root)
- all signing/verification keys are generated deterministically using a PRF of the corresponding node position
  (i.e. WOTS are used to sign always the same verification key)

- signatures are quite long (41KB)
  but key sizes are reasonable (1KB)